Nori

Nori Bridge

Zellic Security Audit — Response

October 15, 2026



Prepared for

Mina Foundation o1Labs

Mina community

Prepared by

Jonathan Kelsey Karol Podufalski Alexander Blair

Deniz Baş Andrija Novakovic

Introduction

Nori is a trust-minimised bridge between Ethereum and Mina that makes the cryptographically verified state of each chain available to applications on the other.

Acknowledgements

We thank the Mina Foundation and o1Labs, and their current and former staff who helped along the way, including but not limited to Coby Durling, Florian Kluge, Matthew Ryan, Dariusz Kijania, Nicole Farrar and Deepthi Kumar, for their generous support throughout this work.

We thank Zellic for their patience and accommodation throughout the engagement, and in particular Pedro Moura, Engagement Manager, and the engineers who conducted the assessment: Katerina Belotskaia, Jiaxin Guan, Hyunsik Jeong, Jaeeu Kim, Jinseo Kim, Malte Leip and Mark Pedron.

We thank Andrija Novakovic, the original author of o1js-blobstream, which we forked into proof-conversion, and Deniz Baş of o1Labs for their contributions to the development of proof-conversion and for their expert technical support.

Finally, we thank the Nori team who saw the audit through from engagement to remediation: Karol Podufalski, Jonathan Kelsey and Alexander Blair.

Scope of the audit

Zellic conducted a security assessment of the Nori Bridge circuits and contracts in the following repositories:

- `proof-conversion` (PC)
- `nori-bridge-sdk` (SDK)
- `nori-bridge-head` (BH)

Zellic provided Nori its findings in *Nori Bridge — Application Security Assessment* (received 18 Sept 2026). The response that follows supplements that report and assumes that the reader is familiar with it.

Each repository has an `AUDIT` branch for posterity, which points to the exact code checkpoint that Zellic reviewed.

Purpose of this document

To provide a detailed summary of our response and all remediative actions conducted in response to Zellic's report. For every finding, we present a brief summary, a comprehensive response, the branch and commits in which remediations were conducted and any pertinent links relating to the issue; enabling readers if they wish to trace and verify responses for themselves.

Remediation team

An audit response and remediation team was assembled to address every finding in Zellic's report. From the Nori core team, Jonathan Kelsey and Karol Podufalski led the investigation of the findings and implemented

their fixes, and Alexander Blair led project management, Zellic engagement and documentation. Andrija Novakovic and Deniz Baş provided cryptography and mathematics support.

Summary of findings

Zellic reported 23 findings by impact: 3 Critical, 3 High, 1 Medium, 4 Low and 12 Informational.

Remediation process

The team addressed each finding on a dedicated branch named `FIX/audit-<code>`, where `<code>` is the identifier that Zellic assigned to the finding (for example, `FIX/audit-b1114`). Where findings were closely related, the team addressed them on a single shared branch (for example, `FIX/audit-d3034-a3850`).

Each finding response follows the same commit pattern:

1. In every **commit 1** (C1), the finding (verbatim), the team's response, and any regression tests exposing it (where deemed necessary) with their results, all documented (in the repository's `CHANGELOG.md`).
2. In every **commit 2** (C2), the team's fix proposal and a report of the regression test results, all documented.
3. In every **commit 3** (C3+), where present and in response to Zellic's feedback: revisions were made to initial fix attempts and changes documented.

Resolution

The assurance process ran from 28 April 2026 to 22 September 2026, 147 days in total. Of the 23 findings, 21 were addressed through remediation commits, one was resolved upstream and one we disagree with. The remediation commits added 99 test cases to proof-conversion, 256 to nori-bridge-sdk and 214 to nori-bridge-head, a sum total of 569 additional checks.

The team merged all of the `FIX/audit-<code>` branches into a single `FIX/audit` branch in each repository (`proof-conversion`, `nori-bridge-sdk`, `nori-bridge-head`), and has since mainlined that work onto develop.

Audit response matrix

#	ID	Severity / Impact	Fix branch(es)	Fix commit(s)
3.1	b1114	Critical / Critical	`PC/FIX/audit-b1114`	C1 PC `01db5085` C2 PC `d9aec26b`
3.2	18fa3	Critical / Critical	`PC/FIX/audit-18fa3-full`	C1 PC `f668ddcc` C2 PC `0352070b` C3 PC `339f3687`
3.3	ec50d	Critical / Critical	`PC/FIX/audit-ec50d`	C1 PC `9bed8442` C2 PC `67209dab`
3.4	d3034	Critical / High	`SDK/FIX/audit-d3034-a3850` `BH/FIX/audit-d3034-a3850`	C1+C2 SDK `0a011766` C1+C2 BH `573dfa9f`
3.5	4279a	High / High	`SDK/FIX/audit-4279a`	C1 SDK `a6454076` C2 SDK `39b7d0a9` C3 SDK `2f903e3c`

#	ID	Severity / Impact	Fix branch(es)	Fix commit(s)
3.6	a3850	High / High	`SDK/FIX/audit-d3034-a3850` `BH/FIX/audit-d3034-a3850`	C1+C2 SDK `0a011766` C1+C2 BH `573dfa9f`
3.7	1eb72	Medium / Medium	`BH/FIX/audit-1eb72`	C1 BH `29fae059` C2 BH `29280373`
3.8	a2090	Medium / Low	`SDK/FIX/audit-a2090` `BH/FIX/audit-a2090`	C1 SDK `769c5eab` C2 SDK `f8270774` C1 BH `acc1997e` C2 BH `58434164`
3.9	41428	Low / Low	`SDK/FIX/audit-41428`	C1 SDK `239a9fdd` C2 SDK `aed341b3` C3 SDK `a3a856eb`
3.10	8ff57	Low / Low	`BH/FIX/audit-8ff57`	C1+C2 BH `67e97e07`
3.11	7f3a1	Low / Low	`SDK/FIX/audit-7f3a1`	C1 SDK `337c11a6` C2 SDK `a5c75a3a`
3.12	1f602	Informational / Informational	`PC/FIX/audit-1f602`	C1 PC `a087f65e` C2 PC `7d49b9be` C3 PC `b08dae95`
3.13	b165f	Informational / Informational	`PC/FIX/audit-b165f`	C2 PC `df976e46`
3.14	0dd9c	Informational / Informational	`PC/FIX/audit-10b08-0dd9c`	C1 PC `f63dce9d` C2 PC `b4ca3c3e`
3.15	10b08	Informational / Informational	`PC/FIX/audit-10b08-0dd9c-extended`	C1 PC `f63dce9d` C2 PC `b4ca3c3e` C3 PC `874d0457`
3.16	1a697	Informational / Informational	`PC/FIX/audit-1a697-a9dea`	C1+C2 PC `f01d3c9a`
3.17	a9dea	Informational / Informational	`PC/FIX/audit-1a697-a9dea`	C1+C2 PC `f01d3c9a`
3.18	adcd3	Informational / Informational	`PC/FIX/audit-adcd3`	C1 PC `93015b57` C2 PC `4209fb0b`
3.19	af97e	Informational / Informational	`PC/FIX/audit-af97e`	C1+C2 PC `b2406ef6`
3.20	d7ffa	Informational / Informational	—	—
3.21	b8891	Informational / Informational	`PC/FIX/audit-b8891`	C1+C2 PC `8b6b0d40`
3.22	e68c5	Informational / Informational	`PC/FIX/audit-e68c5`	C1+C2 PC `47fcd6d4`
3.23	e4e27	Informational / Informational	`BH/FIX/audit-e4e27`	C1+C2 BH `784e70bd`

Findings closed with rationale

- **3.16 [1a697] Field.toBigInt usage.** Zellic classified `Field.toBigInt()` as a debug-only o1js operation, on the basis of its docstring, and recommended replacing it with `Field.toValue`. We disagree. `Field.toValue` simply calls `Field.toBigInt`, as Zellic acknowledges, so the change would be purely nominal. In o1js,

`toBigInt()` returns a value only for constants or inside a `Provable.witness` or `asProver` block. Called on a variable anywhere else in provable code, it throws during circuit compilation, so misuse fails at compile time and cannot silently alter a circuit, as Zellic's own impact assessment concedes. Each of the five affected call sites is either inside a witness block, with the result constrained back in-circuit through `assertEquals`, or off-circuit. o1Labs confirmed that both uses are safe for production systems. `ForeignField.toBigInt()` carries no such warning, which points to an inconsistency in the o1js documentation, not a safety boundary.

- **3.20 [d7ffa] Usage of @internal o1js APIs.** No published o1js release offered these APIs as public during the remediation period. o1Labs has since removed the `@internal` tags, and the APIs are public from o1js 3.0.0 onwards. In our next release of proof-conversion, we will upgrade directly to o1js 3.1.0, the latest release, which includes this change.
- **bcab0, KZG folding randomiser derivation.** Zellic found that `squeezeRandomForKzg` derives the randomiser differently from the SP1 v6.1.0 Rust verifier. We showed that it matches SP1's Solidity verifier and the upstream gnark template, and Zellic withdrew the finding (§4.4). Zellic's own analysis places the only theoretical weakness in the Rust verifier's approach, at a cost of around 2^{127} hash evaluations. The original finding attributed the shortcoming to proof-conversion's formulation, where it is in fact absent.

Post-remediation actions

- **alloy-trie truncated exclusion proofs.** The proof request queue, introduced to resolve 3.6 [a3850], requires the SP1 guest to verify proofs of exclusion against Ethereum state. alloy-trie versions before 0.9.7 accept truncated exclusion proofs, allowing a prover to falsely prove that queue state is empty. A fix, alloy-rs/trie#134, was proposed on 26 February 2026 but was, at the time, unmerged. At 12:27 UTC on 21 September 2026 we documented the defect and cited the fix in a code comment referencing it. It was merged at 16:20 UTC the same day and released in alloy-trie 0.9.7. As of 15 October 2026, nori-bridge-head uses the vulnerable version, alloy-trie 0.9.5. We will move to alloy-trie 0.9.8, the latest release, in our next release.
- **Dependency upgrades.** Dependency versions were frozen during the audit. In our next release, we will upgrade o1js to 3.1.0, SP1 to 6.9.0 and Helios to 0.12.0 (required for Ethereum's Glamsterdam fork), and apply `npm audit fix` and `cargo` dependency fixes across all repositories.
- **`MAX_BATCH` calibration.** `MAX_BATCH` caps the number of proof request queue entries a single MPT consensus proof can cover, and is derived from `MAX_TREE_DEPTH` as 2^`MAX_TREE_DEPTH`. The current depth of 16 is a known bad value. We will set `MAX_TREE_DEPTH` conservatively, keeping each proof within SP1's memory and compute limits while ensuring sufficient volume.

Finding 3.1 [b1114]

- **Report title:** Aggregation leaf circuit layer1 does not constrain disabled proofs to act as identity, allowing forgery of the SP1 PLONK public inputs
- **Severity:** Critical · **Likelihood:** High · **Impact:** Critical
- **Target:** `proof-conversion::compressor::layer1`
- **Reviewed by Zellic:** Yes

Issue

The compressor wraps the verifier steps in a binary tree of recursive proofs. SP1 PLONK has 24 steps, padded to 32, so four layer1 nodes verify neither side. For those disabled sides nothing forced `publicInput ==`

`publicOutput`, so a prover could pick the values freely and carry arbitrary public inputs up to the root proof. That lets an attacker push attacker-chosen head, state root and deposit roots into the bridge and mint for themselves. The Groth16 path (16 steps, a power of two) was not affected.

Fix branch and commits

proof-conversion · branch `proof-conversion/FIX/audit-b1114` · commits list

- C1 `01db5085` · 2026-05-18

> CHORE: audit b1114 commit 1, regression tests exposing missing identity constraint on disabled layer1 subtrees, three jest suites verifying leftIn == rightOut when verifyLeft/verifyRight disabled (0 pass 3 fail), jest runner added with experimental-vm-modules for o1js ESM compatibility, changelog documenting finding, response, discussion and results.

- C2 `d9aec26b` · 2026-05-18 · **cited in report**

> FIX: audit b1114 commit 2, identity constraints added to layer1node.ts enforcing publicInput == publicOutput on disabled subtrees (2 sites), all regression tests pass (3 pass 0 fail), changelog updated.

Response

The finding is acknowledged. The vulnerability is isolated to the SP1 PLONK path. The Groth16 path uses 16 base zkps (a power of two), so all layer1 nodes have both sides verified and the issue does not apply. The `layer1` circuit (`src/compressor/layer1node.ts`) uses `verifyIf` for conditional verification, but when verification is skipped the proof's `publicInput` and `publicOutput` are completely unconstrained - the prover can supply arbitrary values via dummy `DynamicProof` instances. The higher-layer `node` circuit (`src/compressor/compressor.ts`) always calls `.verify()` unconditionally and is not affected.

The codebase originated from o1js-blobstream, which was built for Groth16 with a power-of-two number of circuits, so every layer1 node verifies both sides and no identity constraints are needed. When the PLONK path was later added (24 steps, padded to 32), the same padding was reused without adding identity constraints for the disabled subtrees, leaving the layer1 dummy proofs free to carry arbitrary public outputs up the tree.

Andrija Novakovic, the original author of the o1js-blobstream compressor, reviewed the finding, confirmed the analysis, and noted that a power-of-two structure is not fundamentally required.

The `node` circuit at layers 2–5 always verifies unconditionally, so the vulnerability window is limited to the layer1 dummy pairs (indices 12–15 in the standard SP1 PLONK configuration).

Commit 1 - Test exposure of the missing identity constraint

- **Regression tests** (`src/compressor/b1114_regression.spec.ts`): added `both sides disabled must reject non-identity dummy proofs`, `left disabled must reject piLeft.publicInput != piLeft.publicOutput`, and `right disabled must reject piRight.publicInput != piRight.publicOutput`. Each test compiles the `layer1` ZkProgram, creates dummy `ZkpProofLeft` / `ZkpProofRight` with attacker-chosen public IO values (A=111, M=222, C=333), attempts to produce a layer1 proof with one or both sides disabled (`verifyLeft` / `verifyRight` = `Bool(false)`), and expects proof generation to be rejected. On unpatched code proof generation succeeds when it should not, causing all three tests to fail.

- **Jest runner** (`package.json`): added `test:jest` script using `--experimental-vm-modules` for o1js ESM compatibility.

Run: `npm run test:jest -- src/compressor/b1114_regression.spec.ts`

Results:

- Regression tests: 3 fail, 0 pass. All three tests resolve instead of rejecting, confirming that `layer1.compute` accepts non-identity dummy proofs and the vulnerability is present.

Commit 2 - Fix applied

- `src/compressor/layer1node.ts`: added two identity constraints after `verifyIf` calls (lines 49-55). `piLeft.publicInput.equals(piLeft.publicOutput).or(verifyLeft).assertTrue()` and the analogous constraint for the right side. When a side is verified (`verify* = true`), the `.or` passes unconditionally. When a side is disabled (`verify* = false`), the prover must supply `publicInput == publicOutput`, forcing the disabled subtree to act as identity.

Results:

- Regression tests: 3 pass, 0 fail. `layer1.compute` now rejects non-identity dummy proofs at proof generation time with a constraint unsatisfied error.

Finding 3.2 [18fa3]

- **Report title:** SP1 public value vk_root is not pinned, allowing forgery of public values
- **Severity:** Critical · **Likelihood:** High · **Impact:** Critical
- **Target:** `proof-conversion::plonk`
- **Reviewed by Zellic:** Yes, commits 1 and 2. Commit 3 addresses Zellic's feedback on commit 2.

Issue

An SP1 PLONK proof exposes five public inputs. The verifier circuits did not pin `pi3` (`vk_root`, which selects the trusted set of recursion verification keys), `pi2` (`exit_code`) or `pi4` (`proof_nonce`). With `vk_root` unpinned, an attacker can supply their own recursion keys and have a proof of false public values accepted. The Groth16 path had the same gap.

Fix branch and commits

proof-conversion · branch `proof-conversion/FIX/audit-18fa3-full` · commits list

- C1 `f668ddcc` · 2026-05-20

> CHORE: audit 18fa3 commit 1, regression tests exposing missing pi2/pi3 constraints, four jest tests per path (PLONK zkp0, Groth16 zkp14) with 2 failing per suite (rogue exit_code and vk_root not rejected, 4 pass 4 fail total), save_sp1_vk_root binary and SP1_VK_ROOT constant added as shared infra, changelog documenting finding, response and results

- C2 `0352070b` · 2026-05-20 · **cited in report**

> FIX: audit 18fa3 commit 2, pi2 (exit_code) and pi3 (vk_root) constraints added to zkp0.ts (PLONK) and zkp14.ts (Groth16), all regression tests pass (8 pass 0 fail), changelog updated

- C3 `339f3687` · 2026-08-13

> FIX: audit 18fa3 commit 3, Groth16VendorBrand (sp1/risc0/snarkjs) added (src/groth/vendor.ts) and threaded through createZkp14 (src/groth/recursion/zkp14.ts), prove_zkps.ts and all three Groth16 plans so zkp14 pi-pinning branches per vendor (SP1 exit_code/vk_root, risc0 control-root/bn254-control-id, snarkjs none) instead of hardcoding SP1's constraints on every Groth16 path, risc0 control-root and bn254-control-id derived and cross-validated against Solidity source and a self-verified real proof in pairing-utils (src/risc0_control_id.rs + save_risc0_vk_root binary, surfaced to TS via src/risc0_control_id.ts / risc0_control_id_v3.0.6.json), non-tautological verifiers added per vendor and proof system (src/risc_zero/verify_risc_zero.ts, src/sp1/verify_sp1_groth16.ts, src/sp1/verify_sp1_plonk.ts, src/snarkjs/verify_snarkjs.ts) each re-deriving the pi-digest from the real public inputs and asserting it equals proof.publicOutput.rightOut while pinning proof.publicOutput.subtreeVkDigest to an independently-known node vk digest, four adjacent e2e specs (src/risc_zero/e2e.spec.ts, src/sp1/e2e_groth16.spec.ts, src/sp1/e2e_plonk.spec.ts, src/snarkjs/e2e.spec.ts) running each real conversion pipeline end to end and checking the output through those verifiers rather than a tautological verify(proof, vk), all four passing, vendor_branch_regression.spec.ts cross-testing real sp1/risc0 proofs against the wrong vendor branches (4 pass), vk-tree user script added (npm run vk-tree, src/build_vk_tree.ts + src/{groth,plonk}/zkp_tree.ts + src/{groth,plonk}/compile_leaf.ts + src/utils/spawn_worker.ts) recomputing a path's subtreeVkDigest straight from the compiled leaf and layer1/node circuits with one subprocess per leaf to bound WASM memory, confirmed digit-for-digit equal to the real e2e output for plonk/risc0/snarkjs, documented in README, jest runner infra fixed so the specs can run (root_dir walks up to the package.json, processPool spawns with explicit env, jest.setup wires LogPrinter), example-generators added as a submodule and example-proofs regenerated

- C4 `03fe1d4a` · 2026-09-17

> FIX: Merge FIX/audit into FIX/audit-18fa3-full, resolving two conflicts from independent audit fixes landing on the same lines. In CHANGELOG.md, the 18fa3 entry (19/5/26) was reunited into a single block and reordered to sit correctly between EC50D (26/5/26) and B1114 (18/5/26), with every entry from both branches preserved byte for byte. In src/plonk/recursion/zkp0.ts, both fixes were kept together, the pi2/pi3 pinning from the 18fa3 fix and the ten G1 on curve checks from the EC50D fix, which are independent constraints with no overlap. Also audited the files both branches touched but that auto merged without conflict, src/groth/recursion/prove_zkps.ts, src/groth/vk.ts, and README.md, and confirmed both sides' changes are present in each with no dropped logic and no stale references to the renamed AuXWitness.loadFromPath API anywhere in the tree.

Response

The finding is acknowledged. The vulnerability affects both the SP1 PLONK and SP1 Groth16 verification paths. The finding describes the PLONK path but the same issue applies to Groth16 where `public_inputs[2..4]` are similarly unpinned.

The unpinned `pi3` (vk_root) is the critical issue and must be fixed in both paths.

For `pi2` (exit_code): the verification math in both paths already constrains the witness values to match the SP1 proof. In PLONK, all five public inputs are hashed into the Fiat-Shamir transcript via `squeezeGamma`, so any change to `pi2` would produce a different `gamma` and the proof would reject. In Groth16, zero-valued inputs are skipped in IC accumulation but claiming a non-zero value would add a term to the PI point and break the pairing check. An explicit `pi2 == 0` constraint will be added as defence-in-depth per the auditor's recommendation.

For `pi4` (proof_nonce): this is a caller-supplied value that can be any value by design. Per the finding, checks on `proof_nonce` are not necessary for soundness.

Commit 1 - Regression tests exposing missing pi2/pi3 constraints

- **`pairing-utils/src/bin/save_sp1_vk_root.rs`**: new binary extracting `VK_ROOT_BYTES` from `sp1-verifier` 6.1.0 as a decimal string, writing `src/sp1_vk_root_v6.1.0.json`. This is the recursion-layer merkle root shared by both the PLONK and Groth16 wrapper paths.
- **`pairing-utils/Cargo.toml`**: added `[[bin]]` entry for `save_sp1_vk_root`.
- **`src/sp1_vk_root_v6.1.0.json`**: generated artifact containing `sp1_vk_root` as a decimal string. Located above `src/plonk/` and `src/groth/` because the constant is shared.
- **`src/sp1_vk_root.ts`**: imports the JSON and exports `SP1_VK_ROOT` as an `FrC` for use in both paths.
- **`src/plonk/recursion/18fa3_regression.spec.ts`**: four tests targeting `zkp0`. Loads the real SP1 PLONK proof from `example-proofs/sp1_plonk_obj_v6.1.0.json` and constructs the accumulator following the same pattern as `src/plonk/recursion/prove_zkps.ts:52-67`. One sanity check test uses correct values and then the subsequent tests target: pi2, pi3 and pi4 modifying a single pi value while keeping the rest real, then calls `zkp0.compute()`.
- **`src/groth/recursion/18fa3_regression.spec.ts`**: four tests targeting `zkp14` with 5 inputs. Loads the real SP1 Groth16 proof from `example-proofs/sp1_groth16_obj_v6.1.0.json` and extracts `public_inputs[0..4]` as FrC values. Same setup as plonk, but calls `zkp14.compute()` instead.

Run:

```
npm run test:jest -- src/plonk/recursion/18fa3_regression.spec.ts
npm run test:jest -- src/groth/recursion/18fa3_regression.spec.ts
```

Results (both suites show the same pattern):

Test	PLONK zkp0	Groth16 zkp14
correct values	pass	pass
rogue pi2 (exit_code)	**FAIL**	**FAIL**
rogue pi3 (vk_root)	**FAIL**	**FAIL**
rogue pi4 (proof_nonce)	pass	pass

4 pass, 4 fail across both suites.

The two failing tests (pi2, pi3) confirm the finding: `zkp0` and `zkp14` accept any value for `exit_code` and `vk_root` because no assertEquals constraint exists. These will pass after commit 2 adds the constraints.

Rogue pi4 passes because `proof_nonce` is caller-supplied by design and requires no constraint.

Commit 2 - Fix applied

- `src/plonk/recursion/zkp0.ts`: two constraints added after the input digest check. `acc.state.pi2.assertEquals(FrC.from(0n))` enforces exit_code is zero. `acc.state.pi3.assertEquals(SP1_VK_ROOT)` pins vk_root to the legitimate SP1 recursion merkle root. `FrC` and `SP1_VK_ROOT` imported.
- `src/groth/recursion/zkp14.ts`: same two constraints added after the pis_hash computation. `full_pis[2].assertEquals(FrC.from(0n))` and `full_pis[3].assertEquals(SP1_VK_ROOT)`. `SP1_VK_ROOT` imported.

Results:

- Regression tests: 8 pass, 0 fail across both suites. Rogue exit_code and vk_root are now rejected at proof generation time in both paths.

Commit 3 - Groth16 public input pinning per vendor

The `CHANGELOG.md` entry for this commit was omitted in error.

Reviewing commit 2, Zellic noted:

> Because the Groth16 and PLONK circuits now pin some of the public values, they are not generic anymore, but specific to SP1. In particular the Groth16 circuit was previously quite generic (e.g. also supported different numbers of public values).

proof-conversion converts Groth16 proofs from SP1, risc0 and snarkjs through one shared circuit, `zkp14`, so commit 2's SP1 constraints applied to all three. A risc0 proof carries its claim digest at `pi2` and `pi3`, which commit 2 required to be `0` and `SP1_VK_ROOT`. A snarkjs circuit with fewer than four public inputs has no `pi3` to pin. This commit makes `zkp14` pin the public inputs according to the vendor that produced the proof. The PLONK path is unchanged, because SP1 is its only source.

- `src/groth/vendor.ts`: new `Groth16VendorBrand` (`sp1`, `risc0`, `snarkjs`). The vendor is supplied by the caller, because it cannot be detected from the proof: a 5-input SP1 proof and a 5-input risc0 proof have the same shape.
- `src/groth/recursion/zkp14.ts`: `createZkp14` takes the vendor and pins:
  - **SP1:** `pi2` (`exit_code`) to `0` and `pi3` (`vk_root`) to `SP1_VK_ROOT`, as in commit 2.
  - **risc0:** `pi0` and `pi1`, the two halves of the recursion control root, and `pi4`, the BN254 control id. These are risc0's equivalent of SP1's `vk_root`.
  - **snarkjs:** nothing, because snarkjs circuits are user-defined and have no vendor-fixed keys. Any supported input count is accepted again.
- `src/compute/plans/{sp1,risc0,snarkjs}/groth16.ts`, `src/groth/recursion/prove_zkps.ts`: each Groth16 plan passes its vendor through to `createZkp14`.
- `pairing-utils/src/risc0_control_id.rs`, `pairing-utils/src/bin/save_risc0_vk_root.rs`: derive the risc0 constants from `risc0-circuit-recursion` 4.0.5, the version risc0 v3.0.6 uses, following the public input layout of `RiscZeroGroth16Verifier.sol`. They are cross-checked against `ControlID.sol` in risc0-ethereum v3.0.1 and against a real, self-verified risc0 proof. They are written to `src/risc0_control_id_v3.0.6.json` and exported by `src/risc0_control_id.ts`.
- `src/groth/recursion/vendor_branch_regression.spec.ts`: compiles `zkp14` for SP1 and for risc0, and checks that a real proof from each vendor is accepted by its own branch and rejected by the other.
- `src/groth/recursion/18fa3_regression.spec.ts`: compiles `zkp14` for SP1.

- **End-to-end specs** (`src/sp1/e2e_plonk.spec.ts`, `src/sp1/e2e_groth16.spec.ts`, `src/risc_zero/e2e.spec.ts`, `src/snarkjs/e2e.spec.ts`): run each real conversion pipeline end to end. The output is checked with a verifier that recomputes the digest of the real public inputs and asserts it equals `proof.publicOutput.rightOut`.
- **`npm run vk-tree`** (`src/build_vk_tree.ts`): recomputes a path's `subtreeVkDigest` from the compiled circuits, documented in the README.

Run:

```
cargo test -p proof-conversion-utils risc0_control_id
npm run test:jest -- src/groth/recursion/vendor_branch_regression.spec.ts
npm run test:jest -- src/sp1/e2e_plonk.spec.ts
npm run test:jest -- src/sp1/e2e_groth16.spec.ts
npm run test:jest -- src/risc_zero/e2e.spec.ts
npm run test:jest -- src/snarkjs/e2e.spec.ts
```

Results:

- Vendor branch regression: 4 pass, 0 fail.
- End-to-end specs: 4 pass, 0 fail.
- `vk-tree` output matches the end-to-end `subtreeVkDigest` for PLONK, risc0 and snarkjs.

Commit 4 - Integration merge

This commit merges `FIX/audit` into `FIX/audit-18fa3-full` so the branch could be merged into `FIX/audit`. It makes no change to the fix. Two conflicts were resolved. In `CHANGELOG.md`, the 18fa3 entry was rejoined into a single block in date order. In `src/plonk/recursion/zkp0.ts`, both the `pi2` / `pi3` pinning from this finding and the G1 on-curve checks from 3.3 [ec50d] were kept, since they are independent constraints.

Finding 3.3 [ec50d]

- **Report title:** Missing on-curve and subgroup checks in the Groth16 recursion verifier
- **Severity:** Critical · **Likelihood:** High · **Impact:** Critical
- **Target:** `proof-conversion::groth::recursion`
- **Reviewed by Zellic:** Yes

Issue

The Groth16 recursion verifier consumes prover-supplied curve points but never checked they are on the curve, or, for G2, in the correct prime-order subgroup (which on BN254 G2 needs a separate check). If the bridge is configured to use the Groth16 path, this allows forged public values with the same end result as 3.1.

Fix branch and commits

proof-conversion · branch `proof-conversion/FIX/audit-ec50d` · commits list

- C1 `9bed8442` · 2026-05-28

> CHORE: audit ec50d commit 1, regression tests exposing missing on-curve and subgroup checks on prover-supplied proof points, groth16 5 suites covering negA C PI via zkp0, B on-curve via zkp6, and

> combined G2 subgroup check via zkp6 (0 pass, 5 fail), plonk 10 suites covering all G1 points via zkp0 (0 pass 10 fail), changelog documenting finding, response, discussion and results.

- C2 `67209dab` · 2026-05-28 · **cited in report**

> FIX: audit ec50d commit 2, G1 on-curve checks added to groth16 zkp0 (negA, C, PI) and plonk zkp0 (all 10 points) via new G1Affine.assertOnCurve(), G2 on-curve check (y^2 = x^3 + b_twist) and G2 subgroup check added to groth16 zkp6 enforcing full 4-term endomorphism relation [6u+2]B + pi(B) - pi^2(B) + pi^3(B) = O, B_TWIST constant added to precomputed.ts, off-circuit G2 subgroup sanity check added to witness_tracker.ts, all regression tests pass (5 groth16 + 10 plonk, 15 pass 0 fail), changelog updated.

Response

We agree with the auditor's findings that all three Groth16 prover-supplied proof points (negA, C on G1; B on G2) enter the recursion circuit as raw field coordinates with no curve equation or subgroup membership enforced in-circuit. After discussion with Andrija Novakovic, we extended the scope to include the PLONK path, which has the same class of vulnerability on its 10 prover-supplied G1 points.

Andrija Novakovic provided a four-step framework for G2 subgroup checking that leverages the existing Miller loop:

> During the BN254 optimal-ate Miller loop, we are not only accumulating the F_{p^{12}} Miller value; we also have a curve-point accumulator (T). The main loop computes the short scalar part, roughly [6u+2]Q, and then the final correction steps add/subtract Frobenius images of Q. So at the end, T is checking an endomorphism relation of the form
>
> [6u+2]Q + pi(Q) - pi^2(Q) + pi^3(Q) = O
>
> up to the exact sign/convention used by the implementation.
>
> On G_2[r], Frobenius acts like scalar multiplication by the corresponding Frobenius eigenvalue, so this endomorphism relation is zero modulo r. Therefore, for a valid r-torsion point, the final point accumulator should end at infinity.
>
> So for an untrusted G_2 proof element, the subgroup-check logic can be:
>
> 1. check the point is on the correct twist curve,
> 2. check it is not infinity,
> 3. run the Miller loop including the Frobenius correction steps,
> 4. check that the final curve-point accumulator (T) is infinity.
>
> This serves the same purpose as checking [r]Q = O, but it is cheaper because the large scalar contribution is represented using Frobenius maps and it's already implicit inside the miller loop that we anyway do.

Assessment of each step against the audited circuit (`05d307b7`, `AUDIT`):

1. **Check the point is on the correct twist curve.** Not previously enforced. B is prover-supplied, so $y^2 = x^3 +$ b_twist must be asserted in-circuit. The check adds 209 rows, and zkp6, at 47,322 rows, remains within

its limit.

2. **Check it is not infinity.** Not required. Affine coordinates cannot encode the point at infinity, so a malicious prover cannot supply it.

3. **Run the Miller loop including the Frobenius correction steps.** Already performed. zkp0–zkp6 track T alongside f, and the Frobenius corrections are applied at the end of zkp6.

4. **Check that the final curve-point accumulator (T) is infinity.** Not previously enforced, but straightforward to add. T and the Frobenius corrections already exist in zkp6, but the code discards T without checking it; the last operation is `assert_is_line(T, pi_2_B)`, with no subsequent addition. The fix advances T past `pi_2_B`, computes `pi_3_B = frobenius(pi^2(B))`, and asserts T = −`pi_3_B` (same x, negated y).

The PLONK path does not require these steps, because its G2 points (g2, tau) are hardcoded verification-key constants, precomputed into JSON at build time.

BN254 G1 has prime order r, so any point on the curve is in the subgroup. A single `assertOnCurve()` call per point (125 rows each, via the bn254 `createForeignCurve`) is therefore sufficient. This applies to both the Groth16 path (negA, C, PI) and the PLONK path (l_com, r_com, o_com, qcp_0_wire, grand_product, h0, h1, h2, batch_opening_at_zeta and batch_opening_at_zeta_omega).

Commit 1 - Regression tests exposing missing on-curve and subgroup checks

- **Groth16 regression tests** (`src/groth/ec50d_regression.spec.ts`): 5 tests covering all Groth16 prover-supplied proof points. Three G1 on-curve tests parse a real proof from `src/groth/example_jsons/`, tamper the y coordinate of a single point (+1 mod p, verified off-curve), construct a WitnessTracker, and expect proof generation via the real ZkProgram to be rejected. One G2 on-curve test tampers B.y.c0 and expects zkp6 to reject. One combined G2 subgroup test exercises the endomorphism identity from steps 3-4 above: the Miller loop's T accumulator evaluates [6u+2]B + pi(B) - pi^2(B) + pi^3(B); for B in G2[r] this equals O (T reaches infinity), for B outside G2[r] it does not. The test constructs a point on E'(Fp2) outside G2[r] via Fp2 Tonelli-Shanks and expects zkp6 to reject it, then runs valid proof B through zkp6 and expects acceptance. Both assertions are in one test so it fails on every broken state: missing check (bad B accepted), wrong endomorphism relation (valid B rejected), or correct check (both pass). Tests: `zkp0 must reject off-curve negA`, `zkp0 must reject off-curve C`, `zkp0 must reject off-curve PI`, `zkp6 must reject off-curve B` (G2 on-curve check), `zkp6 subgroup check must reject bad B and accept valid B`.
- **PLONK regression tests** (`src/plonk/ec50d_regression.spec.ts`): 10 tests covering all PLONK prover-supplied G1 points. Each test constructs a real Accumulator from a hardcoded hex proof, tampers the y coordinate of a single point (+1 mod p, verified off-curve), and expects proof generation via PLONK zkp0 to be rejected. Tests: `zkp0 must reject off-curve l_com`, `zkp0 must reject off-curve r_com`, `zkp0 must reject off-curve o_com`, `zkp0 must reject off-curve qcp_0_wire`, `zkp0 must reject off-curve grand_product`, `zkp0 must reject off-curve h0`, `zkp0 must reject off-curve h1`, `zkp0 must reject off-curve h2`, `zkp0 must reject off-curve batch_opening_at_zeta`, `zkp0 must reject off-curve batch_opening_at_zeta_omega`.

Run:

```
GROTH16_VK_PATH=./src/groth/example_jsons/vk.json npm run test:jest -- src/groth/ec50d_regression.spec.ts
npm run test:jest -- src/plonk/ec50d_regression.spec.ts
```

Results:

- Groth16 regression tests: 5 fail, 0 pass. All five tests resolve instead of rejecting, confirming that zkp0 and zkp6 accept off-curve and off-subgroup proof points.
- PLONK regression tests: 10 fail, 0 pass. All ten tests resolve instead of rejecting, confirming that PLONK zkp0 accepts off-curve proof points.

Commit 2 - Fix applied

- `src/ec/index.ts`: added `assertOnCurve()` method to `G1Affine`, delegates to `bn254.assertOnCurve()`.
- `src/towers/precomputed.ts`: added `B_TWIST` constant (3/(9+u) in Fp2) for G2 twist curve equation.
- `src/groth/recursion/zkp0.ts`: added G1 on-curve checks for negA, C, PI.
- `src/groth/recursion/zkp6.ts`: added G2 on-curve check (y^2 = x^3 + b_twist) and G2 subgroup check. After the existing pi(B) and -pi^2(B) Frobenius corrections, T is advanced past pi_2_B and checked against -pi_3_B (same x, negated y), enforcing the full 4-term endomorphism relation [6u+2]B + pi(B) - pi^2(B) + pi^3(B) = O.
- `src/groth/witness_tracker.ts`: added off-circuit G2 subgroup check as dev sanity mirror of the in-circuit check in zkp6.
- `src/plonk/recursion/zkp0.ts`: added G1 on-curve checks for all 10 prover-supplied points.

Results:

- Groth16 regression tests: 5 pass, 0 fail.
- PLONK regression tests: 10 pass, 0 fail.

Finding 3.4 [d3034]

- **Report title:** Compromised admin is not prevented from minting or unlocking illegitimately
- **Severity:** Critical · **Likelihood:** Low · **Impact:** High
- **Target:** `NoriTokenBridge (Mina), NoriTokenBridge (Ethereum)`
- **Reviewed by Zellic:** Yes

Issue

The intended design was that a compromised admin could at worst brick the bridge. In practice the admin can update commitments that must stay updatable (the accepted guest program, the proof-conversion keys, the Helios store hash). These commitments cannot be frozen: the guest program changes as Helios follows Ethereum consensus upgrades, the proof-conversion keys rotate with major SP1 and o1js upgrades, and the Helios store hash must be replaceable when the store or its serialisation changes with Helios or the Ethereum consensus specification. Mina protocol hard forks can also require these commitments to be updated. These are constraints imposed by the systems the bridge depends on, not choices Nori made. Each commitment is a route to getting an update accepted with a deposit root of the admin's choosing. That means minting unbacked tokens on Mina and draining the locked ETH. Nori documented the admin trust assumptions (a FROST multisig quorum). Nori also removed the `genesisRoot` state, which did not restrict admin powers as intended.

Fix branch and commits

nori-bridge-sdk · branch `nori-bridge-sdk/FIX/audit-d3034-a3850` · commits list

- C1+C2 `0a011766` · 2026-08-26 · **cited in report**

> FIX: address audit finding a3850 (deposit root not constrained to be complete): adds the NoriProofRequestQueue contract with end-to-end queue-cursor tracking through the Mina bridge and o1js-zk-utils, replacing the prover-chosen deposit-slot proof. Address audit finding d3034 (compromised admin not prevented from minting or unlocking illegitimately): drops the non-binding genesisRoot state from the Mina bridge and proof byte layout. Updates CHANGELOG.md documenting the admin trust assumptions, and DEPLOYMENT.md and the Ethereum README to drop the removed genesis-root deploy inputs.

nori-bridge-sdk · branch `nori-bridge-sdk/FIX/audit-d3034-a3850-changelog` · commits list

- C1+C2 `27489b74` · 2026-08-26 · *squashed into* `0a011766`

> CHORE: add CHANGELOG.md for d3034 and a3850

nori-bridge-head · branch `nori-bridge-head/FIX/audit-d3034-a3850` · commits list

- C1+C2 `573dfa9f` · 2026-08-26 · **cited in report**

> FIX: address audit finding a3850 (deposit root not constrained to be complete): replaces the prover-chosen deposit-slot proof with an on-chain, append-only proof request queue that the guest is required to drain in order. Address audit finding d3034 (compromised admin not prevented from minting or unlocking illegitimately): removes the non-binding genesis_root commitment from ProofOutputs and ConsensusProofOutputs. Updates CHANGELOG.md with the responses for both findings.

Response — nori-bridge-sdk

The finding is correct, including that the list is not exhaustive. We are taking the first of the two recommended paths: documenting the trust assumptions that govern the admin role.

Three on-chain commitments have to remain updatable for the bridge to stay operable. `noriHeliosProgramPi0` and `proofConversionPO2` pin the accepted guest program and the recursion verification keys; the guest program changes as the Helios light client evolves, and the conversion keys rotate on major SP1 upgrades. `latestHeliusStoreInputHash*` pins the Helios store, which must be replaceable because the store and/or its serialisation can change - that depends on Helios internals that we inherit, as well as changes to Ethereum consensus layer spec that Helios itself inherits. An extended finality failure on Ethereum may itself require a forced store reconstruction. Freezing any of the three would turn a routine upgrade into a redeployment and state migration of the whole bridge.

Each of those commitments is also, as the finding sets out, a route by which a compromised admin could have an `update` accepted that carries a deposit root of their choosing, and mint against it. The bridge consequently does not guarantee that a compromised admin can only halt it; it can mint unbacked tokens and drain the locked pool with current design of Aligned Layer (proposed AVS for the Mina-to-Ethereum bridge).

What constrains the role is procedural rather than cryptographic. The admin is FROST-based multisig, so a change to any of these values requires commitment from a quorum of the signers. The project has a governance process for approving upgrades, and the multisig is held by a set of parties that are expected to be

independent and to follow that process. The trust assumption is that the multisig signers will not collude to approve an upgrade that would allow them to mint unbacked tokens or unlock illegitimately.

Two admin surfaces named in the finding are not live routes to minting. `updateVerificationKey` cannot succeed: the bridge account is deployed with `setVerificationKey` set to `impossibleDuringCurrentVersion()` and `setPermissions` set to `impossible()`, so the account's own verification key is fixed for the current protocol version and the permission cannot be relaxed. `canChangeAdmin` returns `Bool(false)`, and `canMint` is gated on the single-use `mintLock` flag that `noriMint` clears immediately before calling `token.mint`, not on an admin signature. The `FungibleToken` verification key is governed by that token's own deploy-time `allowUpdates` setting.

`genesis_root` is a guest input. It is passed to `verify_update` and `verify_finality_update`, which use it to derive the BLS signing domain; it is not compared against the store, and the store is bound instead by the store-hash chain. It is not committed as a public output anymore (as it wasn't fit for purpose as envisaged) and the corresponding on-chain field and assertion are removed.

As part of a solution to this finding, a standalone document is being prepared to record the trust assumptions that govern the admin role, and to describe the procedures that are expected to be followed to maintain the bridge's integrity. This document will serve as a reference for users and developers, clarifying the limits of what a compromised admin can do and the safeguards in place to prevent abuse.

Changes

- **`genesisRoot` removed** (`contracts/mina/src/NoriTokenBridge.ts`): the `@state(Field) genesisRoot` field, its doc comment, the `genesisRoot` deploy prop on `NoriTokenControllerDeployProps`, the assignment in `deploy()`, and the `update()` assertion against the proof's committed genesis root are all deleted. `ethVerify` no longer returns `ethGenesisRootBytes`.
- **Public-output layout** (`o1js-zk-utils/src/ethVerifier.ts`, `utils.ts`): `genesisRoot` dropped from the `EthInput` struct, from the byte assembly and from `decodeConsensusMptProof`'s offsets. Combined with the queue changes recorded above under Finding a3850, the committed public outputs move from 228 to 220 bytes.
- **Helper removed** (`o1js-zk-utils/src/utils.ts`): `extractGenesisRootFromSP1Proof`, which decoded the genesis root from a proof and returned its Poseidon hash, has no remaining consumer. `extractEthProofQueueAddressFromSP1Proof` replaces `extractEthTokenBridgeAddressFromSP1Proof` for the address the proof now anchors on.
- **Deploy pipeline** (`contracts/ethereum/bin/preDeploy.ts`, `contracts/mina/src/env.ts`, `contracts/mina/src/bin/`): `fetchGenesisValidatorsRoot` and the `ETH_CONSENSUS_RPC` requirement are removed from the pre-deploy step, which no longer queries the beacon chain. `NORI_ETH_GENESIS_ROOT` is dropped from the environment and from the Mina deploy arguments, replaced by `NORI_ETH_PROOF_QUEUE_ADDRESS`.
- **Documentation**: the doc comment on the removed `genesisRoot` field from Mina contract, which described it as an immutable chain anchor preventing store-hash rotation from redirecting the bridge. `contracts/DEPLOYMENT.md` and the Ethereum README drop the genesis-root inputs, the pre-deploy record and the ledger row, and carry the proof request queue address in their place. The admin trust assumptions set out in the response above are recorded in this entry, which is the reference for them until they are carried into the deployment documentation.
- **Corresponding changes to bridge-head**: the changelog entry for this finding is added to the nori-bridge-head repo at `nori-bridge-head/CHANGELOG.md`

Results

- Ethereum contract suite: **136 passing** — `npm test` in `contracts/ethereum`.

- Cross-language vectors: **139 passing** — `npm test -- -t "Proof request queue cross-language vectors"` in `o1js-zk-utils`, including the round-trip of the 220-byte encoding and rejection of a wrong-length buffer.
- No behavioural test accompanies the trust statement itself; it records an accepted assumption, not a code constraint.

Response — nori-bridge-head

The finding is correct, including that the list is not exhaustive. We are taking the first of the two recommended paths: documenting the trust assumptions that govern the admin role.

Three on-chain commitments have to remain updatable for the bridge to stay operable. `noriHeliosProgramPi0` and `proofConversionPO2` pin the accepted guest program and the recursion verification keys; the guest program changes as the Helios light client evolves, and the conversion keys rotate on major SP1 upgrades. `latestHeliusStoreInputHash*` pins the Helios store, which must be replaceable because the store and/or its serialisation can change - that depends on Helios internals that we inherit, as well as changes to Ethereum consensus layer spec that Helios itself inherits. An extended finality failure on Ethereum may itself require a forced store reconstruction. Freezing any of the three would turn a routine upgrade into a redeployment and state migration of the whole bridge.

Each of those commitments is also, as the finding sets out, a route by which a compromised admin could have an `update` accepted that carries a deposit root of their choosing, and mint against it. The bridge consequently does not guarantee that a compromised admin can only halt it; it can mint unbacked tokens and drain the locked pool with current design of Aligned Layer (proposed AVS for the Mina-to-Ethereum bridge).

What constrains the role is procedural rather than cryptographic. The admin is FROST-based multisig, so a change to any of these values requires commitment from a quorum of the signers. The project has a governance process for approving upgrades, and the multisig is held by a set of parties that are expected to be independent and to follow that process. The trust assumption is that the multisig signers will not collude to approve an upgrade that would allow them to mint unbacked tokens or unlock illegitimately.

Two admin surfaces named in the finding are not live routes to minting. `updateVerificationKey` cannot succeed: the bridge account is deployed with `setVerificationKey` set to `impossibleDuringCurrentVersion()` and `setPermissions` set to `impossible()`, so the account's own verification key is fixed for the current protocol version and the permission cannot be relaxed. `canChangeAdmin` returns `Bool(false)`, and `canMint` is gated on the single-use `mintLock` flag that `noriMint` clears immediately before calling `token.mint`, not on an admin signature. The `FungibleToken` verification key is governed by that token's own deploy-time `allowUpdates` setting.

`genesis_root` is a guest input. It is passed to `verify_update` and `verify_finality_update`, which use it to derive the BLS signing domain; it is not compared against the store, and the store is bound instead by the store-hash chain. It is not committed as a public output anymore (as it wasn't fit for purpose as envisaged) and the corresponding on-chain field and assertion are removed.

As part of a solution to this finding, a standalone document is being prepared to record the trust assumptions that govern the admin role, and to describe the procedures that are expected to be followed to maintain the bridge's integrity. This document will serve as a reference for users and developers, clarifying the limits of what a compromised admin can do and the safeguards in place to prevent abuse.

Changes

- **`ProofOutputs`** (`nori-primitives/src/types.rs`): `genesis_root` removed from the committed outputs and from `to_bytes` / `from_bytes`. Together with the queue changes recorded under Finding a3850, the encoding moves from 228 to 220 bytes.

- `ConsensusProofOutputs` (`nori-primitives/src/types.rs`): `genesis_root` removed and `output_block_number` added, mirroring `ProofOutputs`; the encoding moves from 176 to 152 bytes.
- `consensus_program` and `consensus_mpt_program` (`nori-program/src/consensus.rs`): neither commits `genesis_root`. Both continue to accept it as an input and pass it to `verify_update` and `verify_finality_update` for the BLS signing domain. The doc blocks and output tables are updated to match.
- **Downstream messages** (`nori/src/bridge_head/api.rs`, `notice_messages.rs`): `genesis_root` dropped from `ProofMessage` and from the job-succeeded transition notice, since it is no longer a proof output.
- **Documentation**: the guest doc blocks in `nori-program/src/consensus.rs` describing the committed outputs no longer list `genesis_root`. The admin trust assumptions set out in the response above are recorded in this entry, which is the reference for them until they are carried into the future deployment documentation.
- **Corresponding changes to the SDK**: the changelog entry for this finding is added to the SDK repo at `nori-bridge-sdk/CHANGELOG.md`

Results

- `nori-primitives`: **6 passing** — `cargo test -p nori-sp1-helios-primitives`. The public-output encoding is covered by a round-trip test and by an offset test asserting the byte positions the destination chain's verifier reads, both of which move with the layout. The shared byte-encoding vectors are rendered by a Rust test and asserted by the Mina-side circuit, so a divergence between the two implementations fails on one side rather than producing proofs that verify inconsistently.
- No behavioural test accompanies the trust statement itself; it records an accepted assumption rather than a code constraint.

Finding 3.5 [4279a]

- **Report title:** Unvalidated oldestAction in dispatchAndEvict can permanently brick ETH to Mina minting
- **Severity:** High · **Likelihood:** High · **Impact:** High
- **Target:** `bridge-sdk::mina::NoriTokenBridge`
- **Reviewed by Zellic:** No

Issue

The Mina bridge keeps a rolling window of the last 32 deposit roots. When the window is full, `dispatchAndEvict` advanced `windowStart` using an `oldestAction` value supplied by the caller, without checking it. A wrong value moves `windowStart` off the real action chain, after which `noriMint` finds no actions and ETH-to-Mina minting stops permanently.

Fix branch and commits

nori-bridge-sdk · branch `nori-bridge-sdk/FIX/audit-4279a` · commits list

- C1 `a6454076` · 2026-06-10

> CHORE: audit 4279a commit 1, regression tests exposing the issue of corrupting the internal state of reducer starting hash for when sliding window is filled by passing a bogus `oldestAction`

- C2 `39b7d0a9` · 2026-06-10 · **cited in report**

> FIX: audit 4279a commit 2, dispatchAndEvict no longer trusts a caller-supplied `oldestAction` — it derives the oldest action in-circuit Removes the `oldestAction` param from update/adminSetDepositRoot and the getOldestActionForEviction helper, updates call sites, regenerates the bridge VK, and adapts the regression test to the single-arg API (now passes).

- C3 `2f903e3c` · 2026-06-10

> CHORE: audit 4279a commit 3, re-disable the test-only `adminSetDepositRoot` so it stays out of the production contract (re-comment its integration-spec callers, skip the eviction-witness regression suite with a re-enable note), regenerate the bridge VkData/VkHash to reflect the contract without it

Response

The finding is correct. `dispatchAndEvict` advances `windowStart` using the caller-supplied `oldestAction` without constraining it to the real oldest action in the window. When the window is full, a caller can pass any value, moving `windowStart` to an action-state hash that lies off the real action chain. Subsequent `noriMint` calls fetch actions `fromActionState: windowStart`, which then resolves to nothing, disabling minting. As with a2090, testing is bolstered first (commit 1) to expose the issue, then the fix applied (commit 2). The intended fix removes the `oldestAction` parameter and derives the oldest action in-circuit via the reducer, so `windowStart` can only ever advance to a real point on the chain.

Commit 1 - Test exposure of eviction witness integrity

- **Regression test** (`contracts/mina/src/tests/unit/4279a.evictionWitness.reggression.spec.ts`): added a self-contained LocalBlockchain test (`proofsEnabled: false`) that deploys the bridge, fills the deposit-root window to `maxWindow` via the test-only `adminSetDepositRoot` (**enabled for testing the regression ONLY** - will be removed), then dispatches one more deposit root with a deliberately bogus `oldestAction`. It asserts the window still resolves from `windowStart` (holds `maxWindow` roots, including the new one) and that the user can still `noriMint`. The test-only `adminSetDepositRoot` method it relies on is enabled in the same commit.

Results:

- Regression test: fails on the current contract. The bogus `oldestAction` poisons `windowStart`; `fetchActions({ fromActionState: windowStart })` throws `getActions: fromActionState not found`, so the window reads 0 roots (expected 32) and the mint flow is unreachable — confirming the finding.

Commit 2 - Fix applied

- `dispatchAndEvict` **derives the oldest action in-circuit** (`contracts/mina/src/NoriTokenBridge.ts`): the caller-supplied `oldestAction` parameter is removed. The oldest action is now found by running `reducer.reduce` over the current window (`getActions({ fromActionState: windowStart })`) and latching the first action. o1js's reducer asserts the chain `windowStart -> ... -> account.actionState` matches the actions it yields, so the captured oldest is provably the real one and `windowStart` can only ever advance to a real point on the action chain. The new root is dispatched after the reduce so it is not pulled into the eviction scope.
- `oldestAction` **parameter dropped from** `update` **and** `adminSetDepositRoot` (`contracts/mina/src/NoriTokenBridge.ts`) and the now-unused `getOldestActionForEviction` helper

removed (`contracts/mina/src/NoriTokenBridge.utils.ts`). Call sites updated: `proofSubmitter.ts` , `workers/tokenBridgeWorker/worker.ts` , `workers/tokenBridgeTester/worker.ts` , and the integration specs.
- **Bridge verification key regenerated** (`contracts/mina/src/integrity/NoriTokenBridge.VkData.json` , `NoriTokenBridge.VkHash.json`): the `update` circuit changed.
- **Regression test updated to the single-argument API** (`contracts/mina/src/tests/unit/4279a.evictionWitness.reggression.spec.ts`): with no `oldestAction` to supply, the test now confirms a full-window eviction keeps the window healthy and mintable. **NOTE: this regression will pass by definition of the fix, as the oldestAction is no longer provided but deterministically derived in provable way**

Results:

- Regression test: passes. After filling the window to `maxWindow` and dispatching one further root, the window still resolves from `windowStart` (holds `maxWindow` roots, including the new one) and the user mints successfully. `windowStart` can no longer be poisoned by the caller.

Commit 3 - Restore test-only method and regenerate integrity data

- `adminSetDepositRoot` **re-commented** (`contracts/mina/src/NoriTokenBridge.ts`): the test-only seeding method enabled in commit 1 is disabled again so it is not part of the production contract. Its callers in `NoriTokenBridge.full.local.integration.spec.ts` are re-commented, and the eviction-witness regression suite (`contracts/mina/src/tests/unit/4279a.evictionWitness.reggression.spec.ts`) is `describe.skip` -ed with a note documenting how to re-enable it.
- **Bridge verification key regenerated** (`contracts/mina/src/integrity/NoriTokenBridge.VkData.json` , `NoriTokenBridge.VkHash.json`): reflects the production contract without `adminSetDepositRoot` .

Results:

- Regression suite is `describe.skip` -ed by default (depends on the test-only `adminSetDepositRoot`); re-enable per the in-file note to run it.

Finding 3.6 [a3850]

- **Report title:** The committed deposit root is not constrained to be complete
- **Severity:** High · **Likelihood:** Medium · **Impact:** High
- **Target:** `bridge-head::nori-program::mpt`
- **Reviewed by Zellic:** Yes

Issue

Each update commits a deposit root built from `lockedTokens` entries, but the prover chose which keys to include, so nothing forced the root to contain every deposit. A censoring, buggy or offline operator can leave deposits out. Because each update must advance the slot and binds to the previous head, a front-runner can also pre-empt the legitimate update and the mints that depend on it. Nori's fix is a substantial redesign around a new on-chain `NoriProofRequestQueue` that the guest must drain in order.

Fix branch and commits

nori-bridge-sdk · branch `nori-bridge-sdk/FIX/audit-d3034-a3850` · commits list

- C1+C2 `0a011766` · 2026-08-26 · **cited in report**

> FIX: address audit finding a3850 (deposit root not constrained to be complete): adds the NoriProofRequestQueue contract with end-to-end queue-cursor tracking through the Mina bridge and o1js-zk-utils, replacing the prover-chosen deposit-slot proof. Address audit finding d3034 (compromised admin not prevented from minting or unlocking illegitimately): drops the non-binding genesisRoot state from the Mina bridge and proof byte layout. Updates CHANGELOG.md documenting the admin trust assumptions, and DEPLOYMENT.md and the Ethereum README to drop the removed genesis-root deploy inputs.

nori-bridge-head · branch `nori-bridge-head/FIX/audit-d3034-a3850` · commits list

- C1+C2 `573dfa9f` · 2026-08-26 · **cited in report**

> FIX: address audit finding a3850 (deposit root not constrained to be complete): replaces the prover-chosen deposit-slot proof with an on-chain, append-only proof request queue that the guest is required to drain in order. Address audit finding d3034 (compromised admin not prevented from minting or unlocking illegitimately): removes the non-binding genesis_root commitment from ProofOutputs and ConsensusProofOutputs. Updates CHANGELOG.md with the responses for both findings.

Response — nori-bridge-sdk

The finding is correct and the recommendation is accepted. The guest no longer accepts a prover-chosen set of storage slots. The deposit set is now an on-chain, append-only **proof request queue** on Ethereum (`NoriProofRequestQueue`), which the guest is required to drain in order.

`verify_storage_slot_proofs` is replaced by `verify_queue`. The queue holds a monotonic `head` counter and a mapping of numbered request records; the destination chain holds a `queueCursor` recording how many requests have been settled. From those two values the batch is fully determined:

- the start is the cursor, which the destination chain asserts against its own stored value,
- `head` is read out of the queue's storage by Merkle-Patricia proof against the execution state root the same proof commits,
- the size is `min(head - cursor, MAX_BATCH)`, and a witness whose entry count disagrees is rejected with `BatchSizeMismatch`,
- each entry's five storage words are located from its index by Solidity's mapping and struct-member rules, not read from the witness.

Requests that form a batch supplied by the prover have to be derived from on-chain data. The all-zero root is reachable only when `head == cursor`, and that equality is itself proven — at genesis by an exclusion proof that the head slot is absent from the storage trie.

Completeness is total rather than best-effort: every entry in the range contributes exactly one leaf. A populated slot is pinned by an inclusion proof of its true value; an empty slot, or a target account that does not exist, by an exclusion proof, contributing a leaf with value zero. In a Merkle-Patricia trie "holds zero" and "absent from the trie" are the same fact, and for a given root and key exactly one of the two proofs can verify, so the claim is pinned in either direction. A junk request therefore proves as zero and the queue advances past it — it can neither stall the queue nor be silently dropped. A proof that verifies as neither aborts the entire run: a committable failure outcome would reintroduce exactly the omission this finding describes.

This also addresses the footgun raised in the recommendation. Completeness is enforced **incrementally**, not all-at-once: a batch is the outstanding backlog, capped at `MAX_BATCH`, and anything beyond that cap drains across consecutive updates, in order, with no entry skipped. `MAX_BATCH` currently is set to 2^16, the same as the Merkle tree's leaf limit. 2^16 is feasible in the o1js circuits but this value is a parameter that must be lower based on what is possible for SP1 circuit to handle (given memory constraints, circuit size etc).

Forcing a large batch is bounded by what enqueueing costs. Every `requestProof` writes three to five cold storage words and pays `proofRequestQueueFee` on top, a fee that accrues to the protocol rather than being burned. That fee is the economic guard against spamming the queue: entries cannot be added cheaply (fee monitored and controlled by the admin), and an attacker pays the protocol for the proving work each one causes.

On the preemption scenario, a frontrunner must resume from the same cursor and carry the same entries, so a competing proof can no longer advance the head while omitting deposits. On the data-availability point the audit places out of scope, the queue is append-only and never deleted, so the leaf set of any committed batch is reconstructible from Ethereum state by any party; the proof pipeline additionally publishes the verified request set in cursor order.

Changes

- **`NoriProofRequestQueue.sol`** (`contracts/ethereum/contracts/`): new append-only queue contract. Storage layout is consensus-critical: `head` at slot 0 and the `_requests` mapping at slot 1 are read by the circuit; `proofRequestQueueFee`, `operator`, `feeRecipient` and `accumulatedFees` occupy slots 2-5, below the provable region. Request `i` occupies five consecutive words from `keccak256(abi.encode(i, uint256(1)))`: `target`, `slotKey`, `collectionKeysCount`, `collectionKeys[0]`, `collectionKeys[1]` — one field per word, deliberately unpacked so the layout is respecified identically in Solidity, Rust and TypeScript. Unused key words are never written, so they are absent from the storage trie and read as zero via an exclusion proof. `requestProof(bytes32 slotKey, bytes32[] calldata collectionKeys)` stamps `target = msg.sender`. `MAX_COLLECTION_KEYS = 2`; the fee is bounded by `MAX_PROOF_REQUEST_QUEUE_FEE` (0.05 ETH) and must align to `PROOF_REQUEST_QUEUE_FEE_GRANULARITY_WEI` (10^12). `operator` is expected to be the existing `TimelockController`, so fee and recipient changes inherit its delay.
- **`NoriTokenBridge.sol`** (`contracts/ethereum/contracts/`): holds the queue as `NoriProofRequestQueue public immutable proofQueue`. Immutable rather than storage, so it consumes no slot and `lockedTokens` stays at slot 2 (`LOCKED_TOKENS_SLOT_INDEX`), leaving the layout the circuit reads bit-identical; it also has no setter, so both sides of the bridge pin the same queue at deploy time or not at all. `lockTokens` now enqueues one request per lock, deriving `slotKey = keccak256(abi.encode(codeChallenge, LOCKED_TOKENS_SLOT_INDEX))` and a single collection key `bytes32(codeChallenge)` from the same validated input, so the pairing cannot be forged by the caller.
- **Lock fee is additive** (`contracts/ethereum/contracts/NoriTokenBridge.sol`): `_splitFee` charges the flat queue fee plus the existing percentage rate, with `MIN_FEE_BU` continuing to floor the rate portion only. The queue fee is forwarded to the queue in the same call and the treasury keeps the remainder. New error `FeeExceedsLockAmount` rejects a deposit the combined fee would consume. `MIN_LOCK_AMOUNT_WEI` raised from 0.0001 to 0.001 ETH so the flat component stays a modest share of the smallest deposit. New `previewLock(uint256) → (feeWei, netWei)` view shares `_splitFee` with `lockTokens`, so a quote cannot disagree with what is charged; `calcGrossLockAmount` was reworked to match.
- **Mina `NoriTokenBridge`** (`contracts/mina/src/NoriTokenBridge.ts`): added `@state(UInt64) queueCursor` and `@state(Field) ethProofQueueAddress`. `update` now asserts the proof's committed queue address against `ethProofQueueAddress`, asserts `input.inputQueueCursor == queueCursor`, and on success sets `queueCursor = input.outputQueueCursor`. Both new fields are deploy-time only with no setter, matching the treatment of `ethTokenBridgeAddress`.

- **`noriMint` pins the leaf origin** (`contracts/mina/src/NoriTokenBridge.ts`): the queue is shared between consumer contracts, so the deposit leaf's `target` is now asserted to equal `ethTokenBridgeAddress`. This is where that address is checked — the check moved from `update` to `noriMint`, since the address the proof anchors on is now the queue rather than the bridge.
- **`EthInput` and the proof byte layout** (`o1js-zk-utils/src/ethVerifier.ts`): `contractAddress` replaced by `proofRequestQueueAddress`, and `inputQueueCursor`, `outputQueueCursor` and `outputBlockNumber` appended. The byte assembly is now a single exported helper, `ethInputToBytes`, called by both the `EthVerifier` zkProgram and the Mina contract's `ethVerify`, replacing two separately maintained concatenations of a consensus-critical byte order. `decodeConsensusMptProof` (`o1js-zk-utils/src/utils.ts`) was updated to the new offsets and length.
- **New leaf format** (`o1js-zk-utils/src/VerifiedRequestAttestor.ts`): `VerifiedRequest { target, collectionKeysCount, collectionKeys[2], value }` and `provableRequestLeafHash`, which packs 117 bytes into four field elements — `target` (20 bytes) with `collectionKeysCount` and the leading byte of each of the two keys and the value, then the three 31-byte tails. Hashing the count prevents an unused trailing key, which is zero, from colliding with a request that supplied a zero key. Namespacing the leaf by `target` is what lets `noriMint` reject leaves enqueued by another consumer. The superseded `ContractDeposit` and `provableStorageSlotLeafHash` are deleted rather than kept as deprecated aliases, and `contracts/mina/src/depositAttestation.ts` now takes the single definitions from `o1js-zk-utils` rather than keeping its own copies.
- **Cross-language test vectors** (`o1js-zk-utils/src/test-vectors/proof-request-queue/`): three fixtures generated by the Rust implementation and vendored here — `request-leaf-vectors.json` (Poseidon leaf packing), `proof-outputs-vectors.json` (the public-output byte encoding) and `entry-location-vectors.json` (the storage slot of `head` and of each entry's five words). A divergence between the SP1 guest and the Mina circuit produces proofs that verify on one side and not the other, so these three encodings are pinned rather than independently reimplemented.
- **Commitments regenerated**: `NoriTokenBridge.VkHash.json` / `.VkData.json`, `EthVerifier.VkHash.json` and `nori-sp1-helios-program.pi0.json`, covering the circuit and guest changes recorded here and under Finding d3034. `noriHeliosProgramPi0` is admin-pinned on-chain, so deploying this needs the admin-gated update d3034 documents.
- **Test-example proofs regenerated**: the example proof series and both `sp1-mpt-proof` fixtures are updated to the current guest, the previous ones carrying the old 228-byte outputs. The current fixture carries a non-empty batch, so the cross-language pipeline is exercised against real request data rather than an empty root.
- **Deploy pipeline** (`contracts/ethereum/tasks/deploy.ts`): the queue is deployed before the bridge, since the bridge takes its address as an immutable constructor argument, and its operator is set to the bridge operator. `NORI_ETH_PROOF_QUEUE_ADDRESS` is written to `.env.nori-eth-token-bridge` and consumed by the Mina deploy. New optional `NORI_ETH_BRIDGE_PROOF_REQUEST_QUEUE_FEE_WEI`, validated against the ceiling and granularity before anything is deployed. New tasks `setProofRequestQueueFee`, `withdrawProofRequestQueueFees` and `previewFees`; `getFeeInfo` now reports the effective lock fee as the queue fee plus the rate.

Results

- Ethereum contract suite: **136 passing** — `npm test` in `contracts/ethereum`.
- Cross-language vectors: **139 passing** — `npm test -- -t "Proof request queue cross-language vectors"` in `o1js-zk-utils`. Covers the leaf hash against every one of the 125 Rust vectors, the count-versus-zero-key disambiguation, target namespacing, the byte encoding in both directions, and rejection of a wrong-length public-output buffer.

- The layout the circuit depends on has a dedicated consensus-critical block, asserting by raw storage read that `head` is at slot 0, that an entry occupies five words at its index-derived location, that an unused collection-key word is never written and reads zero, and that configuration stays in slots 2-5. Two further tests assert `setOperator` and `withdrawFees` leave the provable slots untouched. The block also cross-checks the shared entry-location vectors, so the slot derivation is verified against the Rust implementation rather than only against itself.

Response — nori-bridge-head

The finding is correct and the recommendation is accepted. The guest no longer accepts a prover-chosen set of storage slots. The deposit set is now an on-chain, append-only **proof request queue** on Ethereum (`NoriProofRequestQueue`), which the guest is required to drain in order.

`verify_storage_slot_proofs` is replaced by `verify_queue`. The queue holds a monotonic `head` counter and a mapping of numbered request records; the destination chain holds a `queueCursor` recording how many requests have been settled. From those two values the batch is fully determined:

- the start is the cursor, which the destination chain asserts against its own stored value,
- `head` is read out of the queue's storage by Merkle-Patricia proof against the execution state root the same proof commits,
- the size is `min(head - cursor, MAX_BATCH)`, and a witness whose entry count disagrees is rejected with `BatchSizeMismatch`,
- each entry's five storage words are located from its index by Solidity's mapping and struct-member rules, not read from the witness.

Requests that form a batch supplied by the prover have to be derived from on-chain data. The all-zero root is reachable only when `head == cursor`, and that equality is itself proven — at genesis by an exclusion proof that the head slot is absent from the storage trie.

Completeness is total rather than best-effort: every entry in the range contributes exactly one leaf. A populated slot is pinned by an inclusion proof of its true value; an empty slot, or a target account that does not exist, by an exclusion proof, contributing a leaf with value zero. In a Merkle-Patricia trie "holds zero" and "absent from the trie" are the same fact, and for a given root and key exactly one of the two proofs can verify, so the claim is pinned in either direction. A junk request therefore proves as zero and the queue advances past it — it can neither stall the queue nor be silently dropped. A proof that verifies as neither aborts the entire run: a committable failure outcome would reintroduce exactly the omission this finding describes.

This also addresses the footgun raised in the recommendation. Completeness is enforced **incrementally**, not all-at-once: a batch is the outstanding backlog, capped at `MAX_BATCH`, and anything beyond that cap drains across consecutive updates, in order, with no entry skipped. `MAX_BATCH` currently is set to 2^16, the same as the Merkle tree's leaf limit. 2^16 is feasible in the o1js circuits but this value is a parameter that must be lower based on what is possible for SP1 circuit to handle (given memory constraints, circuit size etc).

Forcing a large batch is bounded by what enqueueing costs. Every `requestProof` writes three to five cold storage words and pays `proofRequestQueueFee` on top, a fee that accrues to the protocol rather than being burned. That fee is the economic guard against spamming the queue: entries cannot be added cheaply (fee monitored and controlled by the admin), and an attacker pays the protocol for the proving work each one causes.

On the preemption scenario, a frontrunner must resume from the same cursor and carry the same entries, so a competing proof can no longer advance the head while omitting deposits. On the data-availability point the audit places out of scope, the queue is append-only and never deleted, so the leaf set of any committed batch is

reconstructible from Ethereum state by any party; the proof pipeline additionally publishes the verified request set in cursor order.

Changes

- **`verify_queue`** (`nori-program/src/mpt.rs`): replaces `verify_storage_slot_proofs`. Takes the execution state root and the queue witness, and returns the output cursor and the requests root. Verifies the queue account, then the `head` slot, then bounds the cursor, then derives the batch, then each entry's five words at index-derived locations, then each distinct target account and the requested slot beneath it, and finally folds one leaf per entry into the Merkle tree.
- **Inclusion / exclusion handling** (`nori-program/src/mpt.rs`): `verify_storage_word` selects the required proof from the claimed value — a non-zero value requires an inclusion proof of its RLP encoding, a zero value requires an exclusion proof. `verify_account` takes `Option<TrieAccount>`: `Some` requires an inclusion proof and promotes the account's storage root, `None` requires an exclusion proof and yields `EMPTY_ROOT_HASH`, under which only zero-valued slot proofs can verify.
- **Error surface** (`nori-program/src/mpt.rs`): `MptError` now carries `CursorAheadOfHead`, `BatchSizeMismatch`, `MissingTargetWitness` and `MissingSlotWitness` for the queue-specific failures. Account-proof failures are branded by a `ProvenAccount` selector into `InvalidProofRequestQueueAccountProof` and `InvalidTargetAccountProof`, so a failure identifies which account it concerns without comparing addresses.
- **Queue witness types** (`nori-primitives/src/types.rs`): `QueueStorage`, `QueueEntryProof`, `TargetStorageProof`, `TargetSlotProof` and `VerifiedRequest` added; `ProofInputs.contract_storage` replaced by `queue_storage`. `StorageSlot`, `ContractStorage` and `VerifiedContractStorageSlot` are removed.
- **Storage layout module** (`nori-primitives/src/storage_layout.rs`): new home for the queue layout constants (`QUEUE_HEAD_STORAGE_INDEX`, `QUEUE_REQUESTS_STORAGE_INDEX`, `QUEUE_ENTRY_WORDS`, `MAX_COLLECTION_KEYS`) and the Solidity slot arithmetic (`storage_slot_of_index`, `mapping_entry_location`, `struct_word_slot`, `word_of_address`, `word_of_b256`). These are shared by the guest, which derives the keys it verifies, and the host, which fetches those same keys — so there is one definition per rule rather than two that can drift.
- **Public outputs** (`nori-primitives/src/types.rs`): `ProofOutputs` gains `input_queue_cursor`, `output_queue_cursor` and `output_block_number`; `contract_address` becomes `proof_request_queue_address` (the proof now anchors on the queue, not the bridge) and `verified_contract_storage_slots_root` is renamed `verified_requests_root`. With the `genesis_root` removal recorded separately under Finding d3034, the encoding moves from 228 to 220 bytes. `output_block_number` is committed so that any party can identify the block every storage read was taken at and reconstruct the batch independently.
- **Leaf format** (`nori-hash/src/merkle_poseidon_fixed.rs`): `hash_request_leaf` replaces `hash_storage_slot`, packing 117 bytes into four field elements — `target` (20 bytes) together with `collection_keys_count` and the leading byte of each collection key and of the value, then the three 31-byte tails. The count is hashed so that an unused trailing key, which is zero, cannot collide with a request that supplied a zero key. Namespacing each leaf by `target` is what allows a consumer to reject leaves enqueued by a different contract. `MAX_BATCH` lives alongside `MAX_TREE_DEPTH`, from which it derives.
- **Guest wiring** (`nori-program/src/consensus.rs`): `consensus_mpt_program` calls `verify_queue` and commits the queue address, both cursors and the requests root. `execution_state_root` and `output_block_number` are read from a single `store.finalized_header.execution()` call on the post-apply finalised header, so the committed pair cannot disagree about which block the storage reads describe.
- **Host witness assembly** (`nori/src/rpcs/execution/http.rs`): `_prepare_consensus_mpt_proof_inputs` no longer scans `TokensLocked` events to choose slots. It reads `head`, derives the batch and every entry location, then issues one `eth_getProof` against the queue covering the head slot and all entry words, and one per distinct target address. Every read is pinned to the window's output block. An empty batch costs

two RPC calls and still proves `head == cursor`. Account existence is decided by verifying the inclusion proof and then the exclusion proof locally rather than inferring it from the response shape, because clients disagree on how an absent account is represented.

- **RPC resilience** (`nori/src/rpcs/execution/http.rs`): `NORI_EXECUTION_CHUNK_LIMIT` (default 100) caps how many storage keys go into a single `eth_getProof`; requests are chunked and retried with exponential backoff, replacing a hardcoded chunk size.
- **Cursor plumbing** (`nori/src/bridge_head/`): the queue cursor is carried end to end — `BridgeHead` tracks it, `advance` and `run` take it, `AdvanceMessage` and the checkpoint persist it (with a `serde` default so checkpoints written before the queue still load), and the started, job-created, job-succeeded, job-failed and advanced notices all report it. `ProofMessage` and the job-succeeded notice additionally publish `verified_requests`, the committed leaf set in cursor order.
- **Environment** (`nori/src/contract.rs`, `.env.example`, `README.md`): `NORI_TOKEN_BRIDGE_ADDRESS` becomes `NORI_PROOF_QUEUE_ADDRESS`, since the address the proof anchors on is now the queue. The event-scanning helpers and the `nori-contract-bindings` crate they were the last consumer of are deleted.
- **Cross-language test vectors** (`test-vectors/proof-request-queue/`): three fixtures rendered by Rust tests and vendored into the SDK — the Poseidon leaf packing, the public-output byte encoding, and the storage locations of `head` and of each entry's five words. These three encodings must agree bit-for-bit between the SP1 guest and the Mina circuit, so they are pinned rather than reimplemented independently on each side.

Results

- Guest: **40 passing** — `cargo test -p nori-sp1-helios-program --lib`. `nori-hash`: **20 passing** — `cargo test -p nori-hash` (14 unit, 1 integration, 5 doctests).
- Adversarial coverage for `verify_queue` (`nori-program/src/mpt.rs`) is built on fixture tries constructed in-process with `alloy-trie`, requiring no network. It covers the cases where the distinction matters: a populated slot claimed as zero, an empty slot claimed non-zero, an empty slot correctly claimed zero, zero-valued entry words, an absent target account claimed absent, a live account claimed absent, an absent account with a non-zero claim, and the genesis case of `head == 0` yielding an empty batch and the zero root. Batch derivation is covered for a cursor ahead of head and for an entry count that disagrees with the derived batch, along with duplicate-slot deduplication.
- Exclusion proofs are exercised across all three trie shapes an absence can take — an empty branch child, another key's leaf, and a diverging extension node — the last being a historically error-prone corner of Merkle-Patricia verifiers. Each test asserts the node shape its fixture actually produces, so a change to the fixture keys fails rather than silently dropping a shape from the coverage.
- The leaf packing is additionally verified byte by byte: each of the 117 payload bytes is routed through `pack_request_leaf_fields` and checked against the documented field layout by independent arithmetic, so a transposed byte cannot pass unnoticed.

Finding 3.7 [1eb72]

- **Report title:** Missing in-circuit head-advancement and checkpoint-slot assertions
- **Severity:** Medium · **Likelihood:** Low · **Impact:** Medium
- **Target:** `bridge-head::nori-program::consensus`
- **Reviewed by Zellic:** No

Issue

Upstream sp1-helios asserts in-circuit that each new finalised head is later than the previous one and sits on a checkpoint slot (`slot % 32 == 0`). Nori's guest program dropped both. Head advancement is still enforced by the Mina contract, but the checkpoint check exists only in the honest prover's off-chain code, so a malicious prover could commit a non-checkpoint head.

Fix branch and commits

nori-bridge-head · branch `nori-bridge-head/FIX/audit-1eb72` · commits list

- C1 `29fae059` · 2026-07-08

> CHORE: audit 1eb72 commit 1, regression test exposing consensus_mpt_program accepting non-checkpoint slots (output_slot % 32 != 0), fixture generator in nori-test-fixtures captures real ProofInputs at slot 10650047 (% 32 == 31), test asserts program rejects non-checkpoint slots and currently fails confirming the vulnerability, changelog documents verbatim finding and response noting permanent DOS severity and repeatable bricking via permissionless update().

- C2 `29280373` · 2026-07-08 · **cited in report**

> FIX: audit 1eb72 commit 2, consensus_mpt_program rejects non-checkpoint output slots (output_slot % 32 != 0) with NonCheckpointOutputSlot error, checked in step 7 immediately after output_slot is captured in step 6 once updates and finality_update have both been applied, regression test passes, changelog updated.

Response

The severity is higher than the finding suggests. The finding describes a "temporary denial of service" but the impact is a permanent denial of service that is repeatable even after recovery.

The `update()` method on the Mina `NoriTokenBridge` contract is permissionless. Anyone who can produce a valid SP1 proof can call it. The contract advances `latestHead` to the proof's `outputSlot` and updates the store hash chain (`latestHeliusStoreInputHashHighByte`, `latestHeliusStoreInputHashLowerBytes`) to the proof's `outputStoreHash`. Neither the ZK program (`consensus_mpt_program`) nor the contract enforce `outputSlot % 32 == 0`.

If any party submits a proof whose `outputSlot` is a non-checkpoint slot, the honest operator cannot produce a valid next proof from that slot. This does not require malicious intent; anyone running their own operator implementation without the off-chain `% 32` guard would trigger it. When `bootstrap_from_slot` is called, it derives a checkpoint hash from the block at that slot and passes it to the beacon chain's `getLightClientBootstrap` endpoint, which returns "LC bootstrap unavailable" because bootstrapping is only supported for checkpoint slots. The operator cannot reconstruct a Helios store rooted at that slot, so it cannot produce a proof whose `inputStoreHash` matches the now-committed store hash on-chain. The bridge is bricked.

Recovery requires the admin key to call `updateStoreHash()` (`NoriTokenBridge.ts:686`) to manually set the store hash to a valid checkpoint-rooted store. But `update()` remains permissionless and the ZK program still accepts non-checkpoint slots, so the same party can immediately brick the bridge again after recovery.

Missed epoch boundary slots (where the block proposer for a slot at position 0 in the epoch fails to produce a block) occur frequently on mainnet. When this happens, the finalised header points to the last block before the boundary, which is a non-checkpoint slot. An attacker or negligent operator does not need to compromise any proposer; they simply wait for a naturally occurring missed boundary slot and submit a proof during that window.

Commit 1 - Test exposing non-checkpoint slot acceptance

- **`nori-test-fixtures`** (`nori-test-fixtures/`): new workspace crate with a `generate_non_checkpoint_fixture` binary that captures a real `ProofInputs<MainnetConsensusSpec>` for a non-checkpoint finalised slot. Cold starts once via `get_latest_finality_slot_and_store_hash`, then chains `prepare_consensus_mpt_proof_inputs` with `validate=false`, feeding each output slot and store hash back as the next input until a non-checkpoint output slot is observed.
- **`non_checkpoint_proof_inputs.10650047.cbor`** (`nori/tests/data/`): captured fixture, slot 10650047 (% 32 == 31), 113807 bytes.
- **`1eb72_non_checkpoint_regression`** (`nori/tests/1eb72_non_checkpoint_regression.rs`): regression test that deserialises the fixture and passes it to `consensus_mpt_program`. Asserts the program rejects non-checkpoint slots.

Results:

- `1eb72_non_checkpoint_regression`: FAILED. `consensus_mpt_program` accepts slot 10650047 (% 32 == 31) without error, confirming the vulnerability. The program produces a valid `ProofOutputs` with a non-checkpoint `output_slot`, which could be submitted to `NoriTokenBridge.update()` to brick the bridge.

Commit 2 - Fix applied

The constraint is enforced against `output_slot`, i.e. `store.finalized_header.beacon().slot` as it stands after `consensus_mpt_program` has finished applying `updates` and `finality_update`, since `output_slot` is the value committed into `ProofOutputs` and ultimately written on-chain by `NoriTokenBridge.update()`. `store.finalized_header` is not set from a single, fixed source: both `apply_update` (called once per entry in `updates`) and `apply_finality_update` (called for `finality_update`) route through the same helios function, `apply_generic_update` (`helios ethereum/consensus-core/src/consensus_core.rs`), which first decides whether the update qualifies at all:

```
// helios: ethereum/consensus-core/src/consensus_core.rs, apply_generic_update
let should_apply_update = {
    let has_majority = committee_bits * 3 >= S::sync_committee_size() * 2;
    let update_is_newer = update_finalized_slot > store.finalized_header.beacon().slot;
    let good_update = update_is_newer || update_has_finalized_next_committee;
    has_majority && good_update
};
if should_apply_update {
    apply_update_no_quorum_check(store, update);
}
```

Only if `should_apply_update` is true does it call `apply_update_no_quorum_check`, which re-checks slot progression before actually writing the field:

```
// helios: ethereum/consensus-core/src/consensus_core.rs, apply_update_no_quorum_check
if update_finalized_slot > store.finalized_header.beacon().slot {
    store.finalized_header = update.finalized_header.clone().unwrap();
}
```

This is the only line in helios that assigns `store.finalized_header`. `verify_update` / `verify_finality_update` establish that an update is cryptographically valid; neither establishes that `should_apply_update` or this second slot check will hold for it. So whether processing a given entry in `updates`, or `finality_update`, changes `store.finalized_header` at all, and to what, depends on the store's finalised slot as left by whatever was processed immediately before it. After both processing steps, `store.finalized_header.beacon().slot` can therefore end up matching any one of `updates`, `finality_update`, or `input_slot` unchanged, none of which `consensus_mpt_program` can determine ahead of time. That is why the check reads `output_slot` and asserts `output_slot % 32 == 0` only in step 7, after step 6 has captured `store.finalized_header.beacon().slot` post-apply.

- **`ProgramError`** (`nori-program/src/consensus.rs`): added `NonCheckpointOutputSlot { slot: u64 }` variant with display format showing the slot and its `% 32` remainder.
- **`consensus_mpt_program`** (`nori-program/src/consensus.rs`): added step 7 Checkpoint Slot Validation, checking `output_slot % 32 != 0` and returning `NonCheckpointOutputSlot`, immediately after `output_slot` is captured in step 6 (post-apply). Docstring operations and error-condition lists renumbered and updated accordingly.
- The check is not added to `consensus_program` because it is only used as an off-chain dry run in `prepare_consensus_mpt_proof_inputs`, gated by the `validate` flag. The existing off-chain `% 32` check in `mod.rs` already handles the `validate=true` case. `consensus_mpt_program` is the ZK circuit entrypoint (`nori-program/src/main.rs`), so enforcing the constraint there makes it impossible to produce a valid proof for a non-checkpoint slot.

Results:

`cargo test -p nori --test 1eb72_non_checkpoint_regression`

- `1eb72_non_checkpoint_regression`: PASSED. `consensus_mpt_program` rejects slot 10650047 (% 32 == 31) with `NonCheckpointOutputSlot` error.

This test is not sufficient proof that the check is anchored to the right value. Its fixture has an empty `updates` array, so `finality_update`'s own (pre-apply) slot and the true post-apply `output_slot` are numerically identical (10650047 in both), and the test cannot tell a check on one from a check on the other. A fixture that could tell them apart would need an `updates[i]` entry that advances `store.finalized_header` to a non-checkpoint slot, followed by a `finality_update` that fails to override it.

Sepolia's live missed-slot rate is about 3.3% per epoch boundary (measured via `light-sepolia.beaconcha.in`, 50 missed slots over 1505 slots), which is what let the existing fixture be captured in about 3.2 hours of polling. But `updates` is only non-empty when a proof crosses a sync-committee period boundary (8192 slots, about 27.3 hours), so on average about 13.65 hours pass before there is even an `updates` entry to look at, and getting that entry's own finalised slot to land on non-checkpoint needs roughly 30 such boundaries at the same 3.3% rate, around 34 days of continuous polling. On top of that the update still has to go unoverridden by the corresponding `finality_update`, which depends on operational conditions (which of `all_providers_urls` a call to `multiplex` lands on, node lag, timing skew between the `get_updates` and `finality_update` calls) rather than slot-miss statistics, and we have no measured rate for it. Given all of that, we are documenting this as a known gap rather than waiting for it.

Finding 3.8 [a2090]

- **Report title:** Merkle zero index reversed
- **Severity:** Medium · **Likelihood:** Low · **Impact:** Low
- **Target:** `bridge-sdk::o1js-zk-utils::merkleTree, bridge-head::nori-hash::merkle_poseidon_fixed`
- **Reviewed by Zellic:** No

Issue

`getMerkleZeros` caches roots of all-zero subtrees. The tree builders read that cache in reverse order (an all-padding subtree gets the cached root for the wrong height), giving different roots from any conventional Merkle implementation whenever the tree has an all-padding subtree. It is consistent inside Nori today (TypeScript and Rust share the convention), so the risk is integration: anyone building deposit witnesses with standard Merkle code gets roots that do not match, and their mints fail. Zellic noted the fix has to be applied on both the TypeScript and Rust sides together.

Fix branch and commits

nori-bridge-sdk · branch `nori-bridge-sdk/FIX/audit-a2090` · commits list

- C1 `769c5eab` · 2026-05-15

> CHORE: audit a2090 commit 1, regression tests exposing non-standard Merkle zeros indexing, four independent test.each suites verifying buildMerkleTree and foldMerkleLeft against brute-force and o1js MerkleTree references (8 pass 16 fail), cross-reference script and docs added, unpatched cross-language agreement verified, changelog documenting finding, response, discussion and results.

- C2 `f8270774` · 2026-05-15 · **cited in report**

> FIX: audit a2090 commit 2, zeros[level] corrected to zeros[depth + 1 - level] in buildMerkleTree, foldMerkleLeft, getMerklePathFromLeaves (3 sites), all regression tests pass (24 pass 0 fail), self-consistency and cross-language agreement verified, changelog updated.

nori-bridge-head · branch `nori-bridge-head/FIX/audit-a2090` · commits list

- C1 `acc1997e` · 2026-05-15

> CHORE: audit a2090 commit 1, regression tests exposing non-standard Merkle zeros indexing, two independent reference implementations (brute-force and recursive) verifying build_merkle_tree and fold_merkle_left (8 pass 16 fail), unpatched cross-language agreement verified, changelog documenting finding, response, discussion and results.

- C2 `58434164` · 2026-05-15

> FIX: audit a2090 commit 2, zeros[level] corrected to zeros[depth + 1 - level] in fold_merkle_left, build_merkle_tree, get_merkle_path_from_leaves (3 sites), all regression tests pass (24 checks 24

> pass), self-consistency and cross-language agreement verified, changelog updated.

Response — nori-bridge-sdk

The non-standard indexing is acknowledged. The same reversed indexing exists symmetrically in both the TypeScript (`merkleTree.ts`) and Rust (`merkle_poseidon_fixed.rs`) implementations across all three affected functions: `buildMerkleTree` / `build_merkle_tree`, `foldMerkleLeft` / `fold_merkle_left`, and `getMerklePathFromLeaves` / `get_merkle_path_from_leaves`. Because both producers in this closed system use the same non-standard convention, the computed roots agree across languages for all leaf counts. We do not believe there is a soundness or completeness failure in the deployed system. If the bug were asymmetric, failures would appear at any non-power-of-two count leaving adjacent dummies, the smallest being n=5, then 6, 9, 10, 11, 12, 13. Applying the proposed fix to only one side would introduce the completeness failure described in the report. The mistake cancels out, leaving it safe as written but highly non-standard. It is worth fixing, but needs to be done carefully to avoid regression of the mint function.

After discussion it was noted that the two cited tests are not sufficient to rule out cross-language divergence when run in isolation, as each only checks self-consistency within its own language. This is agreed. The tests were not designed to be used in isolation. They were designed to be used in concert: the raw output from any two of the three test suites (Rust, TypeScript non-provable, TypeScript provable) was compared using an uncommitted comparison script that normalised and diffed leaves and roots line-by-line across languages. An improved version of this script (`o1js-zk-utils/test/cross-reference-roots.sh`) is now committed to nori-bridge-sdk for transparency.

Three test suites cover this code:

1. Rust - `cargo test -p nori-hash test_all_leaf_counts_and_indices_with_build_and_fold` (n_leaves 0-50)
2. TypeScript (non-provable) - `npm run test -- -t "test_all_leaf_counts_and_indices_with_build_and_fold"` (n_leaves 0-50)
3. TypeScript (provable) - `npm run test -- -t "test_all_leaf_counts_and_indices_with_pipeline"` (n_leaves 0-10, truncated for speed; previously run to 50)

This cross-referencing is a sample-based confidence check, not a claim of a completeness proof.

The finding correctly identifies a deviation from the standard Merkle zero-hash convention. While harmless in the current closed two-implementation system, non-standard indexing would be a problem for any future third-party verifier or public auditability tooling that assumes the standard convention. The fix is accepted and will be applied to both sides simultaneously. Testing will be bolstered first (commit 1) to expose the non-standard indexing against independent reference implementations, then the fix applied (commit 2), so that the before and after results can be documented in this summary.

Commit 1 - Test exposure of the non-standard indexing

- **Regression tests** (`o1js-zk-utils/src/merkle-attestor/merkleTree.spec.ts`): added `regression_a2090_bruteforce_reference_buildMerkleTree`, `regression_a2090_bruteforce_reference_foldMerkleLeft`, `regression_a2090_o1js_merkle_tree_reference_buildMerkleTree`, and `regression_a2090_o1js_merkle_tree_reference_foldMerkleLeft` as `test.each` over leaf counts [1, 3, 5, 6, 9, 17] verifying `buildMerkleTree` and `foldMerkleLeft` against two independent reference implementations. The brute-force reference pads with Field(0) and hashes every pair with no zeros cache. The o1js `MerkleTree` reference uses the standard o1js Merkle tree implementation. Neither references the zeros array. Leaf

counts 5, 6, 9, 17 exercise the bug (adjacent dummy nodes at various depths); 1 and 3 are sanity cases where only a lone dummy pairs with a real leaf.
- **Cross-reference script** (`o1js-zk-utils/test/cross-reference-roots.sh`): committed for reproducible cross-language verification of leaves and roots across the Rust, TypeScript non-provable, and TypeScript provable test suites. See `o1js-zk-utils/test/CROSS-REFERENCE-ROOTS.md` for usage.

Results:

- Regression tests: nLeaves 1 and 3 pass (no adjacent dummies, 8 pass). nLeaves 5, 6, 9, 17 fail against both the brute-force and o1js `MerkleTree` references for both `buildMerkleTree` and `foldMerkleLeft` (8 failures per reference, 16 fail, 24 total checks), confirming the non-standard indexing is detectable and diverges from the standard convention.
- Self-consistency (Rust): passes 0-50 leaves.
- Self-consistency (TypeScript non-provable): passes 0-50 leaves.
- Self-consistency (TypeScript provable): passes 0-10 leaves.
- Cross-reference (unpatched Rust vs unpatched TypeScript non-provable): 51 leaf counts, all leaves and roots match. Zero differences.
- Cross-reference (unpatched Rust vs unpatched TypeScript provable): 11 leaf counts (0-10), all leaves and roots match. Zero differences.

Commit 2 - Fix applied

- **`zeros[level]` corrected to `zeros[depth + 1 - level]`** (`o1js-zk-utils/src/merkle-attestor/merkleTree.ts`): three sites patched in `buildMerkleTree` (line 86), `foldMerkleLeft` (line 144), and `getMerklePathFromLeaves` (line 219). When the tree-building loop is at a given `level` counting down from `depth`, the parent node of two dummy children represents an all-zero subtree of height `depth + 1 - level`. The corrected index selects the matching precomputed zero hash from `getMerkleZeros`.

Results:

- Regression tests: 24 pass, 0 fail. All leaf counts [1, 3, 5, 6, 9, 17] now match both the brute-force and o1js `MerkleTree` references for both `buildMerkleTree` and `foldMerkleLeft`.
- Self-consistency (Rust): passes 0-50 leaves.
- Self-consistency (TypeScript non-provable): passes 0-50 leaves.
- Self-consistency (TypeScript provable): passes 0-10 leaves.
- Cross-reference (patched Rust vs patched TypeScript non-provable): 51 leaf counts, 0 leaf mismatches, 0 root mismatches.
- Cross-reference (patched Rust vs patched TypeScript provable): 51 leaf counts checked, 0 root mismatches, 40 leaf mismatches (all MISSING, provable suite only runs 0-10, no data exists for 11-50), 11 overlapping leaf counts all leaves and roots match.
- Cross-reference (patched TypeScript non-provable vs patched TypeScript provable): 51 leaf counts checked, 0 root mismatches, 40 leaf mismatches (all MISSING, provable suite only runs 0-10, no data exists for 11-50), 11 overlapping leaf counts all leaves and roots match.

Response — nori-bridge-head

The non-standard indexing is acknowledged. The same reversed indexing exists symmetrically in both the TypeScript (`merkleTree.ts`) and Rust (`merkle_poseidon_fixed.rs`) implementations across all three affected functions: `buildMerkleTree` / `build_merkle_tree`, `foldMerkleLeft` / `fold_merkle_left`, and `getMerklePathFromLeaves` / `get_merkle_path_from_leaves`. Because both producers in this closed system use the same non-standard convention, the computed roots agree across languages for all leaf counts. We do not

believe there is a soundness or completeness failure in the deployed system. If the bug were asymmetric, failures would appear at any non-power-of-two count leaving adjacent dummies, the smallest being n=5, then 6, 9, 10, 11, 12, 13. Applying the proposed fix to only one side would introduce the completeness failure described in the report. The mistake cancels out, leaving it safe as written but highly non-standard. It is worth fixing, but needs to be done carefully to avoid regression of the mint function.

After discussion it was noted that the two cited tests are not sufficient to rule out cross-language divergence when run in isolation, as each only checks self-consistency within its own language. This is agreed. The tests were not designed to be used in isolation. They were designed to be used in concert: the raw output from any two of the three test suites (Rust, TypeScript non-provable, TypeScript provable) was compared using an uncommitted comparison script that normalised and diffed leaves and roots line-by-line across languages. An improved version of this script (`nori-bridge-sdk/o1js-zk-utils/test/cross-reference-roots.sh`) is now committed to nori-bridge-sdk for transparency.

Three test suites cover this code:

1. Rust - `cargo test -p nori-hash test_all_leaf_counts_and_indices_with_build_and_fold` (n_leaves 0-50)
2. TypeScript (non-provable) - `npm run test -- -t "test_all_leaf_counts_and_indices_with_build_and_fold"` (n_leaves 0-50)
3. TypeScript (provable) - `npm run test -- -t "test_all_leaf_counts_and_indices_with_pipeline"` (n_leaves 0-10, truncated for speed; previously run to 50)

This cross-referencing is a sample-based confidence check, not a claim of a completeness proof.

The finding correctly identifies a deviation from the standard Merkle zero-hash convention. While harmless in the current closed two-implementation system, non-standard indexing would be a problem for any future third-party verifier or public auditability tooling that assumes the standard convention. The fix is accepted and will be applied to both sides simultaneously. Testing will be bolstered first (commit 1) to expose the non-standard indexing against independent reference implementations, then the fix applied (commit 2), so that the before and after results can be documented in this summary.

Commit 1 - Test exposure of the non-standard indexing

- **Regression tests** (`nori-hash/src/merkle_poseidon_fixed.rs`): added `regression_a2090_bruteforce_reference` and `regression_a2090_recursive_reference` tests over leaf counts [1, 3, 5, 6, 9, 17] verifying `build_merkle_tree` and `fold_merkle_left` against two independent reference implementations. The brute-force reference pads with zeros and hashes every pair with no zeros cache. The recursive reference builds the tree top-down, returning `Fp(0)` for empty subtrees. Neither references the zeros array. Leaf counts 5, 6, 9, 17 exercise the bug (adjacent dummy nodes at various depths); 1 and 3 are sanity cases where only a lone dummy pairs with a real leaf.

Results:

- Regression tests: n_leaves 1 and 3 pass (no adjacent dummies, 8 pass). n_leaves 5, 6, 9, 17 fail against both the brute-force and recursive references for both `build_merkle_tree` and `fold_merkle_left` (8 failures per reference, 16 fail, 24 total checks), confirming the non-standard indexing is detectable and diverges from the standard convention.
- Self-consistency (Rust): passes 0-50 leaves.
- Self-consistency (TypeScript non-provable): passes 0-50 leaves.
- Self-consistency (TypeScript provable): passes 0-10 leaves.
- Cross-reference (unpatched Rust vs unpatched TypeScript non-provable): 51 leaf counts, all leaves and roots match. Zero differences.

- Cross-reference (unpatched Rust vs unpatched TypeScript provable): 11 leaf counts (0-10), all leaves and roots match. Zero differences.

Commit 2 - Fix applied

- **`zeros[level]` corrected to `zeros[depth + 1 - level]`** (`nori-hash/src/merkle_poseidon_fixed.rs`): three sites patched in `fold_merkle_left` (line 137), `build_merkle_tree` (line 230), and `get_merkle_path_from_leaves` (line 327). When the tree-building loop is at a given `level` counting down from `depth`, the parent node of two dummy children represents an all-zero subtree of height `depth + 1 - level`. The corrected index selects the matching precomputed zero hash from `get_merkle_zeros`.

Results:

- Regression tests: 2 pass, 0 fail (`regression_a2090_bruteforce_reference`, `regression_a2090_recursive_reference`). All leaf counts [1, 3, 5, 6, 9, 17] now match both the brute-force and recursive references for both `build_merkle_tree` and `fold_merkle_left` (24 checks, 24 pass).
- Self-consistency (Rust): passes 0-50 leaves.
- Self-consistency (TypeScript non-provable): passes 0-50 leaves.
- Self-consistency (TypeScript provable): passes 0-10 leaves.
- Cross-reference (patched Rust vs patched TypeScript non-provable): 51 leaf counts, 0 leaf mismatches, 0 root mismatches.
- Cross-reference (patched Rust vs patched TypeScript provable): 51 leaf counts checked, 0 root mismatches, 40 leaf mismatches (all MISSING, provable suite only runs 0-10, no data exists for 11-50), 11 overlapping leaf counts all leaves and roots match.
- Cross-reference (patched TypeScript non-provable vs patched TypeScript provable): 51 leaf counts checked, 0 root mismatches, 40 leaf mismatches (all MISSING, provable suite only runs 0-10, no data exists for 11-50), 11 overlapping leaf counts all leaves and roots match.

Finding 3.9 [41428]

- **Report title:** In-flight mints are invalidated by the next update
- **Severity:** Low · **Likelihood:** Medium · **Impact:** Low
- **Target:** `bridge-sdk::mina::NoriTokenBridge`
- **Reviewed by Zellic:** No

Issue

The rolling window is meant to give users hours to mint after locking. But `noriMint` pins `windowStart` as a precondition, so any `update` that lands between building a mint proof and its inclusion invalidates that mint. At roughly a 16-minute update cadence and ~2 minutes to prove and submit, a noticeable share of mints would fail and need retrying.

Fix branch and commits

nori-bridge-sdk · branch `nori-bridge-sdk/FIX/audit-41428` · commits list

- C1 `239a9fdd` · 2026-06-19

> CHORE: audit 41428 commit 1, lightnet regression test exposing in-flight mint invalidation - a mint proven against windowStart is rejected when an update slides windowStart before inclusion; enables

> the test-only adminSetDepositRoot. Runs on lightnet because LocalBlockchain does not model the 5-slot actionState precondition window. Fails on the current contract (stale windowStart precondition).

- C2 `aed341b3` · 2026-06-19 · **cited in report**

> FIX: audit 41428 commit 2, noriMint witnesses windowStart instead of pinning it via getAndRequireEquals, so an update between proving and inclusion no longer invalidates an in-flight mint (only the 5-slot-tolerant actionState precondition stays shared with update). Updates all noriMint call sites (tokenBridgeTester + tokenBridgeWorker workers, full.local + main-thread lightnet specs) and the regression test to the 3-arg signature, regenerates the bridge VK. Regression passes green on lightnet; full.lightnet and happyPath.lightnet pass.

- C3 `a3a856eb` · 2026-06-19

> CHORE: audit 41428 commit 3, re-disable the test-only adminSetDepositRoot for production (re-comment its call sites, re-skip the gated noriMint and 41428 regression suites) and regenerate the production bridge VkData/VkHash to reflect the contract without it.

Response

The finding is correct. `noriMint` reads `windowStart` via `getAndRequireEquals()`, which pins `windowStart == W` as a precondition on the mint transaction at prove time. Once the window is full every `update` advances `windowStart`, so an `update` landing between proving and inclusion makes the mint's precondition stale and the network rejects it (~12.5% at the cited cadence). The `actionState` end of the reducer read tolerates the last 5 action states, but `windowStart` is an ordinary state field with no tolerance, so a single update is enough.

Commit 1 - Test exposure of in-flight mint invalidation

- **Regression test** (`contracts/mina/src/tests/41428.inflightMint.lightnet.integration.spec.ts`): added a self-contained lightnet test that fills the deposit-root window to `maxWindow` (honest evictions via the test-only `adminSetDepositRoot`), seeds the user's deposit as the newest window member, then builds **and proves** a mint. One further dispatch (an `update`) slides `windowStart` (with a sanity assert that it moved), after which the already-proven mint is sent; the test asserts it is still accepted and credits the balance. No mempool race is needed — the rejection is a stale precondition, reproduced sequentially (prove → update → send). It runs against lightnet because a real Mina node models the 5-element `actionState` precondition window the fix relies on; LocalBlockchain does not (a probe showed it rejects even a single intervening dispatch), so it cannot demonstrate the green side. The test-only `adminSetDepositRoot` method it relies on is enabled in the same commit.

Results:

- Regression test: fails on the current contract. The intervening `update` moves `windowStart`, and sending the proven mint is rejected on the now-stale `windowStart` precondition (`Account_app_state_precondition_unsatisfied`) — confirming the finding.

Commit 2 - Fix applied

- **`noriMint` witnesses the window start** (`contracts/mina/src/NoriTokenBridge.ts`): instead of reading `windowStart` via `getAndRequireEquals()` (a zero-tolerance precondition), the method takes `windowStartWitness: Field` and uses it as the `getActions` start. The witness is fully constrained without that precondition — `reduce` asserts the chain `windowStartWitness -> ... -> account.actionState` matches the folded actions (a stale/off-chain witness cannot pass), and `maxUpdatesWithActions: maxWindow` bounds it from below. The only shared precondition left with `update` is `account.actionState`, which tolerates the last ~5 action states, so an `update` between proving and inclusion no longer invalidates the mint.
- **Call sites updated** to fetch and pass the witness: `workers/tokenBridgeTester/worker.ts`, `workers/tokenBridgeWorker/worker.ts` (three mint paths), and the integration specs (`full.local`, `main-thread/full.lightnet`, `main-thread/happyPath.lightnet`).
- **Regression test updated** to the witnessed (3-arg) call so it now exercises the fix.
- **Bridge verification key regenerated** (`integrity/NoriTokenBridge.VkData.json`, `NoriTokenBridge.VkHash.json`): the `noriMint` circuit changed.

Results:

- Regression test: passes on lightnet. After the intervening `update` slides `windowStart`, the already-proven mint is accepted (distance 1, within the 5-slot `actionState` tolerance) and credits the balance.
- Tolerance confirmed on lightnet by a probe: a pinned `actionState` survives up to 4 intervening updates (accepted at distances 1 and 4, rejected at 5+); LocalBlockchain rejects even distance 1 — the reason the green side runs on lightnet.
- The worker-driven `full.lightnet` and `happyPath.lightnet` suites pass with the fix.

Commit 3 - Restore test-only method for production

- **`adminSetDepositRoot` re-commented** (`contracts/mina/src/NoriTokenBridge.ts`): the test-only seeding method enabled in commit 1 is disabled again so it is not part of the production contract. Its call sites in `full.lightnet` / `happyPath.lightnet` are re-commented and those `noriMint()` blocks re-`describe.skip`-ed; the `41428.inflightMint.lightnet` regression suite is `describe.skip`-ed (it depends on `adminSetDepositRoot`).
- **Bridge verification key regenerated** (`integrity/NoriTokenBridge.VkData.json`, `NoriTokenBridge.VkHash.json`): reflects the production contract without `adminSetDepositRoot`.

Results:

- The regression suite is `describe.skip`-ed by default (depends on the test-only `adminSetDepositRoot`); re-enable per the in-file note and run against lightnet to reproduce.

Finding 3.10 [8ff57]

- **Report title:** Empty updates response causes get_first_update to panic
- **Severity:** Low · **Likelihood:** Medium · **Impact:** Low
- **Target:** `bridge-head::nori::rpcs::consensus`
- **Reviewed by Zellic:** No

Issue

`get_first_update` takes element 0 of the RPC's update list without checking it is non-empty. An empty response causes a panic during the cold-start procedure.

Fix branch and commits

nori-bridge-head · branch `nori-bridge-head/FIX/audit-8ff57` · commits list

- C1+C2 `67e97e07` · 2026-06-15 · **cited in report**

> FIX: Audit 8ff57, get_first_update no longer panics on empty updates, replaced match block with map_err providing a meaningful error including the period number, an is_empty() guard returning an error, and a guarded get(0).unwrap().clone() that is now unreachable on empty, added defense in depth rationale comments to both get_first_update and prepare_consensus_proof_inputs explaining why empty updates are unexpected in practice, changelog documents the verbatim finding, response with evidence from the consensus spec and Lighthouse and Helios codebases, and the commit details.

Response

The period queried by `get_first_update` is the period of a slot that has already been finalised and bootstrapped from. The call chain is:

1. `bootstrap_from_slot` / `bootstrap_from_checkpoint` gives a `finalized_header` at some slot
2. `calc_sync_period` computes which sync committee period that slot falls in
3. `get_updates(period, 1)` asks the beacon node for light client updates from that period

A sync committee period on mainnet spans 8192 slots (~27 hours). The beacon API endpoint `/eth/v1/beacon/light_client/updates?start_period=P&count=1` returns the best `LightClientUpdate` the node has stored for period P. For a beacon node to have allowed a bootstrap from a finalised checkpoint in period P, that period must be completed or in progress with finality. And a period with finality is expected to produce light client updates, as they are derived from the finality attestations made by that period's sync committee. A beacon node that serves a bootstrap for a period but has zero light client updates for that same period would be unexpected in practice. The consensus spec and major client implementations (including helios, which is our dependency) expect at least one update per period:

- **Consensus spec** (v1.6.1, `specs/altair/light-client/full-node.md` line 171): "Full nodes SHOULD provide the best derivable `LightClientUpdate` … for each sync committee period" (link)
- **Lighthouse** (v8.1.3, `light_client_server_cache.rs` line 195-209): quotes the spec requirement verbatim in a comment and implements it by storing the best update per `sync_period` during block processing (link)
- **Helios** (0.11.1, `consensus.rs` line 460-463): `advance()` calls `get_updates(current_period, 1)` then `updates.get_mut(0).unwrap()`, expecting the 0th update to exist. Empty is treated as "nothing to do" rather than an error state (link)

The only theoretical edge case would be querying a period that is so new that finality has not yet occurred within it, but that cannot happen here because the period is derived from an already-finalised header, not from wall clock time.

In practice, if one has bootstrapped from a finalised slot in period P, the beacon node will have at least one update for period P. The unwrap is technically unclean but not practically reachable.

`get_first_update` is not in the production path and was never planned to be. It is only called from `get_latest_finality_slot_and_store_hash`, which is the test cold start procedure. In production, the initial store hash is generated by a separate script (`nori/bin/generate_initial_store_hash.rs`, see branch `FEAT/inital-store-hash-script`) which mirrors `prepare_consensus_proof_inputs` directly and has its own empty updates guard. The production path (`prepare_consensus_proof_inputs`) already had the correct error handling before this finding was reported.

The bridge head runs containerised. This function was never planned to be in the production path, but if it were, a panic would result in a container restart with no impact on the wider system.

The fix is accepted regardless. Replacing the unwrap with a proper error return is the right thing to do for code quality and auditability.

Commit

- **`get_first_update`** (`nori/src/rpcs/consensus/mod.rs`): replaced the `match` block containing `updates.get_mut(0).unwrap()` with a `map_err` providing a meaningful error message including the period number, an `is_empty()` guard returning `Err(anyhow!("Error updates were missing 0th update."))`, and `updates.get(0).unwrap().clone()` after the guard. The unwrap is now unreachable due to the empty check above it.
- **`prepare_consensus_proof_inputs`** (`nori/src/rpcs/consensus/mod.rs`): updated the comment above the existing `is_empty()` guard from "not sure how to deal with this yet" to "this shouldn't happen but this is defense in depth", and added a rationale comment explaining why empty updates are not expected.

Finding 3.11 [7f3a1]

- **Report title:** Mempool-visible codeChallenge allows griefing of lockTokens callers
- **Severity:** Low · **Likelihood:** Low · **Impact:** Low
- **Target:** `NoriTokenBridge (Ethereum)`
- **Reviewed by Zellic:** No

Issue

On the first `lockTokens` for a `codeChallenge`, the sender became its permanent depositor. Because `codeChallenge` is visible in the mempool, an attacker could front-run with their own lock and cause the honest depositor's transaction to revert. No funds are at risk and the attack costs the attacker their own ETH, so it is griefing only.

Fix branch and commits

nori-bridge-sdk · branch `nori-bridge-sdk/FIX/audit-7f3a1` · commits list

- C1 `337c11a6` · 2026-06-23

> CHORE: audit 7f3a1 commit 1, multiple eth depositors should be able to use the same codeChallenge. Refactored `Should bind Mina account to first depositor and reject others` test to prove the opposite behaviour should be permitted `Should allow different depositors to lock to the same codeChallenge and accumulate correctly`, the modify test fails exposing the current contracts behaviour is

> undesirable. CHANGELOG updated, commit 1 documented, also fixed a type relating on the test `Should set NORI_BRIDE_ZKAPP_ACCT_TOKEN_ID from constructor`

- C2 `a5c75a3a` · 2026-06-23 · **cited in report**

> FIX: audit 7f3a1 commit 2, removed depositKeyToEthAddress mapping, MinaAccountLinkedToDifferentDepositor error and distinct eth depositor per `codeChallenge` validation logic from 'contracts/ethereum/contracts/NoriTokenBridge.sol', updated documentation for 'contracts/mina/src/scram.ts' removing stale reference to depositKeyToEthAddress, updated changelog documenting the changes, all tests now pass.

Response

We agree with the finding and have relaxed the constraint.

The design went through two iterations. In the earlier iteration, codeChallenge was derived from an Ethereum signature and a Mina public key. After the latest redesign, the Mina signature was adopted, which produces a deterministic signature for a given message. The msg.sender binding was left in the contract as a result of cognitive bias carried over from the prior design. After careful review, it was agreed internally that the binding is no longer required.

The auditors confirmed that it can be removed without any impact.

The original constraint existed because codeChallenge previously incorporated both keys, which required an additional constraint to ensure correct accounting for cumulative lockedTokens.

Commit 1 - Regression test exposing the undesirable depositor binding

`contracts/ethereum/test/NoriTokenBridge.ts` contained a test `Should bind Mina account to first depositor and reject others` asserting that a second depositor calling lockTokens with a codeChallenge already used by a different address is rejected with MinaAccountLinkedToDifferentDepositor. Prior to any code change this test passed, confirming the constraint was live. Because we are relaxing that constraint, this passing test here is itself an erroneous behaviour.

`npm test -- --grep "Should bind Mina account to first depositor and reject others"` run in `contracts/ethereum` against the unmodified contract:

Results:

- 1 passing. The test passes against the unmodified contract, confirming the binding is live and that this behaviour is erroneous under the relaxed constraint.
- We also noticed a typo in `Should set NORI_BRIDE_ZKAPP_ACCT_TOKEN_ID from constructor` and corrected that.

We renamed and refactored the test `Should bind Mina account to first depositor and reject others` to assert the opposite `Should allow different depositors to lock to the same codeChallenge and accumulate correctly`. Updating the assertion from "a failure should occur and the contract should revert" - to the contract should permit the action of multiple ETH depositors sharing the same `codeChallenge`.

The full test suite was run to ensure no other regressions:

`npm run test`

Result:

`Should allow different depositors to lock to the same codeChallenge and accumulate correctly` fails with `SolidityError: VM Exception while processing transaction: reverted with an unrecognized custom error (return data: 0x30506bb1)` demonstrating the contract's undesirable rejection of the action - prior to relaxing the constraint.

Commit 2 - Fix applied

- `contracts/ethereum/contracts/NoriTokenBridge.sol`
  - Removed `depositKeyToEthAddress` mapping, which captures the binding from a `codeChallenge` to an ETH address
  - Removed `MinaAccountLinkedToDifferentDepositor` bespoke error.
  - Removed `Enforce one ETH depositor per Mina account` validation logic.
- `contracts/mina/src/scram.ts`
  - Updated documentation to remove references of `depositKeyToEthAddress`

Re-ran the ethereum contract tests:

`cd contracts/ethereum && npm run test`

Result:

62 tests passed, showing that the relaxed constraints allow the modified test to pass.

Finding 3.12 [1f602]

- **Report title:** Function wordToBytes does not guarantee canonicity for large bytesPerWord
- **Severity:** Informational · **Likelihood:** N/A · **Impact:** Informational
- **Target:** `proof-conversion`
- **Reviewed by Zellic:** No

Issue

`wordToBytes` checks that the bytes recombine to the input, but for `bytesPerWord` above ~31 the recombined value can wrap the field modulus, so more than one byte string passes. No current call site is affected (all use 4 or 8 bytes).

Fix branch and commits

proof-conversion · branch `proof-conversion/FIX/audit-1f602` · commits list

- C1 `a087f65e` · 2026-08-06

> CHORE: audit 1f602 commit 1, changelog documenting wordToBytes canonicity bound finding, regression test exposing missing bytesPerWord guard in wordToBytes (3 pass 1 fail), changelog updated.

- C2 `7d49b9be` · 2026-08-06 · **cited in report**

> FIX: audit 1f602 commit 2, compile-time guard added to wordToBytes rejecting bytesPerWord where 2^(8*bytesPerWord) > Field.ORDER, redundant wordToBytes/bytesToWord removed from sha_hash.ts in favor of the guarded copy in utils.ts, all regression tests pass (4 pass 0 fail), changelog updated.

- C3 `b08dae95` · 2026-08-20 · **cited in report**

> FIX: audit 1f602 commit 3, wordToBytesCanonical added to src/sha/utils.ts as a sound sibling of wordToBytes for bytesPerWord > 31, backed by new isCanonicalFieldBytesLE range check against FIELD_PRIME_LE, both exported from src/index.ts and src/index.min.ts alongside wordToBytes, regression tests added proving the canonicity check rejects forged p and p+1 byte arrays that satisfy the old mod-p equality alone, all regression tests pass (11 pass 0 fail), changelog/readme updated.

Response

We agree with the finding, both copies of the wordToBytes function could accept multiple distinct UInt8[] values reconstructing a degenerate word modulo p when 2^(8 * bytesPerWord) > p. Moreover we recognise the code duplication as non optimal.

Commit 1 - Regression test for 1f602

- **Regression test** (`src/sha/1f602_regression.spec.ts`): added `bytesPerWord at the safe bound (2^(8n) <= Field.ORDER) must not throw`, `bytesPerWord one past the safe bound (2^(8n) > Field.ORDER) must throw`, `at the safe bound, wordToBytes round-trips to the original word`, and `default bytesPerWord=8 must not throw`. The safe bound is computed from `Field.ORDER` itself (largest bytesPerWord with `2^(8*bytesPerWord) <= Field.ORDER`) rather than hardcoded, so the test tracks the actual field in use rather than today's specific byte count.

Run: `npm run test:jest -- src/sha/1f602_regression.spec.ts`

Results:

- Regression tests: 1 fail, 3 pass. `wordToBytes` does not throw one byte past the safe bound, confirming the missing guard described in the finding.

Commit 2 - Fix applied

- **`src/sha/utils.ts`**: added the compile-time guard `if (1n << BigInt(8 * bytesPerWord) > Field.ORDER) throw ...` at the top of `wordToBytes`, rejecting any `bytesPerWord` for which the byte range could wrap around the field modulus.
- **`src/sha/sha_hash.ts`**: removed the redundant local copies of `wordToBytes` and `bytesToWord`, now imported from `src/sha/utils.ts` so the guard applies uniformly to every caller.

Run: `npm run test:jest -- src/sha/1f602_regression.spec.ts`

Results:

- Regression tests: 4 pass, 0 fail. `wordToBytes` now throws for any bytesPerWord that would let the byte range wrap around `Field.ORDER`.

Commit 3, `wordToBytesCanonical`, a sound decomposition for `bytesPerWord` greater than 31

Commit 2 makes `wordToBytes` safe by rejecting `bytesPerWord` greater than 31 outright, which is sufficient for the finding as scoped, but it leaves no way to decompose a `Field` into more than 31 bytes at all. A sibling function is added here to give downstream users a known safe option to reach for when they need more than 31 bytes, usable whether inside or outside circuitry, instead of each one rolling its own unconstrained, unsafe decomposition.

- **`src/sha/utils.ts`**:
  - Added `isCanonicalFieldBytesLE(bytes: UInt8[]): Bool`, asserting a little-endian byte array is strictly less than the field prime `p` (Pallas base field), against a new `FIELD_PRIME_LE` constant.
  - Added `wordToBytesCanonical(word: Field, bytesPerWord = 8): UInt8[]`. At `bytesPerWord <= 31` it delegates to `wordToBytes`. Above 31, it performs the same `bytesToWord(bytes).assertEquals(word)` reconstruction check, then additionally asserts `isCanonicalFieldBytesLE(bytes)`. That second check is the fix: the reconstruction equality alone only holds mod `p`, so a prover solving the constraint system directly, not bound to our witness-generation code, could satisfy it with `bytes = word + k*p` for `k >= 1` instead of `word` itself. For example `bytes = p` satisfies the equality for `word = 0` just as validly as `bytes = 0`. The range check rules out every such alternate.
  - `wordToBytesCanonical`, `isCanonicalFieldBytesLE`, `FIELD_PRIME_LE` added to the module's export list.
- **`src/index.ts`** / **`src/index.min.ts`**: `wordToBytesCanonical` and `isCanonicalFieldBytesLE` exported as public API siblings of `wordToBytes`.
- **`src/sha/1f602_regression.spec.ts`**: added a `regression_1f602_wordToBytesCanonical` block:
  - Genuine-witness round-trip at 31 bytes (delegation boundary) and 32 bytes.
  - `isCanonicalFieldBytesLE` accepts the boundary-legal values `0` and `p - 1`.
  - `isCanonicalFieldBytesLE` rejects `p` and `p + 1` even though `bytesToWord` reduces them to `0` and `1`.
  - `FIELD_PRIME_LE` checked against `Field.ORDER`.

Gate cost (`check_wordtobytes_cost.mjs`, `ZkProgram.analyzeMethods`):

- `wordToBytes` at `bytesPerWord` 31: 94 rows.
- `wordToBytesCanonical` at `bytesPerWord` 32: 342 rows.
- The canonicity check adds 248 rows over `wordToBytes`.

Run: `npm run test:jest -- src/sha/1f602_regression.spec.ts`

Results:

- Regression tests: 11 pass, 0 fail.

Finding 3.13 [b165f]

- **Report title:** Usage of Provable.switch without an explicit range constraint on shift_power
- **Severity:** Informational · **Likelihood:** N/A · **Impact:** Informational
- **Target:** `proof-conversion::groth::verifier, proof-conversion::plonk`
- **Reviewed by Zellic:** Yes

Issue

Six `Provable.switch` call sites select a shift factor using `shift_power`, intended to be 0, 1 or 2, but never range-check it. Out-of-range values are only rejected because `Provable.switch` happens to return zero on an all-false

mask and a later assert catches it. No current soundness gap; the risk is a future o1js or code change silently breaking both verifiers.

Fix branch and commits

proof-conversion · branch `proof-conversion/FIX/audit-b165f` · commits list

- C2 `df976e46` · 2026-05-26 · **cited in report**

> FIX: audit b165f, explicit range constraint on shift_power at all 6 Provable.switch call sites (3 groth16, 3 plonk), enforcing shift_power in {0, 1, 2} via polynomial constraint (3 gates per site). Previously only implicitly rejected via Provable.switch returning Fp12.zero() on all-false mask, an undocumented o1js implementation detail, not an API contract

Response

We agree with the finding. The `CHANGELOG.md` entry for this finding was omitted in error. The fix adds an explicit range constraint on `shift_power` at all six `Provable.switch` call sites (three in Groth16, three in PLONK), enforcing `shift_power` $\in \{0, 1, 2\}$ through a polynomial constraint of three gates per site. Previously, out-of-range values were rejected only implicitly, because `Provable.switch` returns `Fp12.zero()` on an all-false mask, an undocumented o1js implementation detail rather than an API contract.

Finding 3.14 [0dd9c]

- **Report title:** Function ArrayListHasher.hash does not enforce configured array length
- **Severity:** Informational · **Likelihood:** N/A · **Impact:** Informational
- **Target:** `proof-conversion::array_list_hasher`
- **Reviewed by Zellic:** Yes

Issue

`ArrayListHasher` is configured for a fixed length, but `hash()` accepts arrays of any length without complaint. Every current caller passes the right length, so this guards against future mistakes that would produce wrong circuits or confusing errors.

Fix branch and commits

proof-conversion · branch `proof-conversion/FIX/audit-10b08-0dd9c` · commits list

- C1 `f63dce9d` · 2026-06-10

> CHORE: audit 10b08 and 0dd9c commit 1, changelog documenting ArrayListHasher duplication (10b08) and missing array length validation (0dd9c), regression tests verifying hash rejects wrong-length arrays (1 pass 2 fail).

- C2 `b4ca3c3e` · 2026-06-10 · **cited in report**

> FIX: audit 10b08 and 0dd9c commit 2, length assertion added to ArrayListHasher.hash() (0dd9c), duplicate ArrayListHasher removed from kzg/structs.ts and replaced with re-export from array_list_hasher.ts (10b08), all regression tests pass (3 pass 0 fail), changelog updated.

Response

We acknowledge the finding. The observation about `Provable.Array`'s `toInput` is correct: it iterates the actual array elements via `.reduce()`, ignoring the declared `n`. `hashPacked` calls `toInput` without calling `check()`, so no length validation occurs at the hashing step.

In current usage, the length is structurally enforced at every call site: inside ZkProgram circuits, `lines_hashes` is declared as `Provable.Array(Field, ATE_LOOP_COUNT.length)` in `privateInputs` and `fromFields` reconstructs exactly that many elements before the method body runs; in the witness trackers, the array is constructed as `new Array(ATE_LOOP_COUNT.length).fill(Field(0n))` and modified in-place; in `open()`, the three input arrays are `Provable.Array`-declared with sizes summing to 65. A wrong-length array cannot reach `hash()` through any current call path, and if one side drifted the `assertEquals` between the witness tracker digest and the circuit digest would catch it. The risk is limited to future refactors where a clear error at the assertion point is preferable to a cryptic `Field.assertEquals` failure downstream.

Commit 1 - Regression test for 0dd9c

- **Regression test** (`src/0dd9c_regression.spec.ts`): added `hash with fewer than n elements must throw`, `hash with more than n elements must throw`, and `hash with exactly n elements must not throw`. The first two tests call `ArrayListHasher.hash()` with arrays of length `n - 1` and `n + 1` respectively and expect an error to be thrown. The third test confirms that an array of exactly `n` elements (65) hashes without error.

Run: `npm run test:jest -- src/0dd9c_regression.spec.ts`

Results:

- Regression tests: 2 fail, 1 pass. The wrong-length arrays do not throw on unpatched code, confirming that `hash()` silently hashes whatever it receives regardless of the declared `n`.

Commit 2 - Fix applied

- **`src/array_list_hasher.ts`**: added `arr.length !== this.n` assertion at the top of `hash()`, throwing with expected and actual lengths if the array size does not match. Since `empty()` and `open()` both route through `hash()`, one assertion covers all three methods.

Run: `npm run test:jest -- src/0dd9c_regression.spec.ts`

Results:

- Regression tests: 3 pass, 0 fail. `ArrayListHasher.hash()` now rejects wrong-length arrays with a descriptive error.

Finding 3.15 [10b08]

- **Report title:** Parallel implementations of shared utilities across Groth16 and PLONK components
- **Severity:** Informational · **Likelihood:** N/A · **Impact:** Informational

- **Target:** `proof-conversion::array_list_hasher, proof-conversion::kzg::structs, proof-conversion::aux_witness, proof-conversion::plonk::aux_witness, proof-conversion::line_parser, proof-conversion::plonk::recursion::line_parser`
- **Reviewed by Zellic:** No

Issue

`ArrayListHasher`, `AuXWitness` / `AuXWitnessType` and `LineParser` each exist as two independent copies (Groth16 and PLONK) with nothing keeping them in sync. No current bug; the risk is the copies drifting apart and the extra maintenance.

Fix branch and commits

proof-conversion · branch `proof-conversion/FIX/audit-10b08-0dd9c` · commits list

- C1 `f63dce9d` · 2026-06-10

> CHORE: audit 10b08 and 0dd9c commit 1, changelog documenting ArrayListHasher duplication (10b08) and missing array length validation (0dd9c), regression tests verifying hash rejects wrong-length arrays (1 pass 2 fail).

- C2 `b4ca3c3e` · 2026-06-10 · **cited in report**

> FIX: audit 10b08 and 0dd9c commit 2, length assertion added to ArrayListHasher.hash() (0dd9c), duplicate ArrayListHasher removed from kzg/structs.ts and replaced with re-export from array_list_hasher.ts (10b08), all regression tests pass (3 pass 0 fail), changelog updated.

proof-conversion · branch `proof-conversion/FIX/audit-10b08-0dd9c-extended` · commits list

- C1 `f63dce9d` · 2026-06-10

> CHORE: audit 10b08 and 0dd9c commit 1, changelog documenting ArrayListHasher duplication (10b08) and missing array length validation (0dd9c), regression tests verifying hash rejects wrong-length arrays (1 pass 2 fail).

- C2 `b4ca3c3e` · 2026-06-10 · **cited in report**

> FIX: audit 10b08 and 0dd9c commit 2, length assertion added to ArrayListHasher.hash() (0dd9c), duplicate ArrayListHasher removed from kzg/structs.ts and replaced with re-export from array_list_hasher.ts (10b08), all regression tests pass (3 pass 0 fail), changelog updated.

- C3 `874d0457` · 2026-06-16 · **cited in report**

> FIX: audit 10b08 extended, AuXWitness consolidated into single shared implementation (Plonk superset moved to src/aux_witness.ts, Groth call sites renamed to loadFromPath), LineParser consolidated (Plonk stateful version removed, Plonk zkp13-16 and witness_tracker switched to shared

> stateless src/line_parser.ts, JSON loading centralised in src/plonk/mm_loop/load_lines.ts), changelog updated

Response

We acknowledge the finding. Both classes are functionally identical. `n` is set to `ATE_LOOP_COUNT.length` on both, so the hardcoded variant in `kzg/structs.ts` produces the same results.

Commit 2 - Fix applied

- **`src/kzg/structs.ts`**: removed duplicate `ArrayListHasher` class (10b08), replaced with import and re-export from `src/array_list_hasher.ts`. All Plonk imports (`from '../../kzg/structs.js'`) continue working without changes. Unused imports (`Poseidon`, `Provable`, `ATE_LOOP_COUNT`) removed.

Response — extended

We acknowledge the extended finding. Both duplications follow the same pattern as ArrayListHasher, which was already consolidated under finding 10b08.

- AuXWitness: the Plonk version (src/plonk/aux_witness.ts) is a strict superset of the Groth version (src/aux_witness.ts). The only API difference is the method name: `parse` (Groth) vs `loadFromPath` (Plonk). The Plonk version additionally provides `loadFromJSON`. The struct definition and all field types are identical.
- LineParser: the Groth version (src/line_parser.ts) is a stateless utility with `static parse(from, to, lines)` and `static frobenius_lines(lines)`. The Plonk version (src/plonk/recursion/line_parser.ts) is a stateful class that bundles JSON loading with the same slicing logic. The core function `ateCntSlice` is identical. The Plonk version also duplicates its JSON loading with src/plonk/recursion/witness_tracker.ts, which independently loads the same g2_lines.json and tau_lines.json at module scope.

Commit - Fix applied

- **`src/aux_witness.ts`**: replaced with the Plonk superset version (moved from src/plonk/aux_witness.ts). Exports `loadFromPath` (renamed from Groth's `parse`) and `loadFromJSON`. Import path for towers/fp12 updated to reflect new location.
- **`src/plonk/aux_witness.ts`**: removed, moved up to src/aux_witness.ts.
- **`src/groth/e2e_test.ts`**, **`src/groth/recursion/prove_zkps.ts`**, **`src/groth/ec50d_regression.spec.ts`**: Groth call sites renamed from `.parse()` to `.loadFromPath()`.
- **`src/plonk/verifier.ts`**, **`src/plonk/e2e_test.ts`**, **`src/plonk/e2e_verify.ts`**, **`src/plonk/recursion/prove_zkps.ts`**: Plonk import paths repointed from `./aux_witness.js` or `../aux_witness.js` to `../aux_witness.js` or `../../aux_witness.js`.
- **`src/plonk/mm_loop/load_lines.ts`**: new file, single location for loading and parsing g2_lines.json and tau_lines.json into G2Line arrays.
- **`src/plonk/recursion/line_parser.ts`**: removed, replaced by shared src/line_parser.ts and src/plonk/mm_loop/load_lines.ts.
- **`src/plonk/recursion/witness_tracker.ts`**: removed duplicate JSON loading, imports g2_lines and tau_lines from ../mm_loop/load_lines.js.
- **`src/plonk/recursion/zkp13.ts`**, **`zkp14.ts`**, **`zkp15.ts`**, **`zkp16.ts`**: switched from Plonk's stateful LineParser to shared stateless LineParser from src/line_parser.ts, line data imported from ../mm_loop/load_lines.js. Frobenius calls in zkp16 updated to use `LineParser.frobenius_lines()`.

Finding 3.16 [1a697]

- **Report title:** Function Field.toBigInt is a debug-only o1js operation but is used outside debug contexts
- **Severity:** Informational · **Likelihood:** N/A · **Impact:** Informational
- **Target:** `proof-conversion`
- **Reviewed by Zellic:** No — we disagree (see Findings closed with rationale)

Issue

Zellic classified `Field.toBigInt()` as a debug-only o1js operation, on the basis of its docstring, and recommended replacing it with `Field.toValue`. o1Labs, the maintainers of o1js, confirmed that `toBigInt()` is safe for production use in witness blocks and off-circuit code, and o1js itself prevents misuse by throwing at compile time if it is called on a variable in provable code. `Field.toValue` simply calls `Field.toBigInt`. See Findings closed with rationale.

Fix branch and commits

proof-conversion · branch `proof-conversion/FIX/audit-1a697-a9dea` · commits list

- C1+C2 `f01d3c9a` · 2026-07-02 · **cited in report**

> FIX: audit 1a697 and a9dea, dead code G2Affine.add removed from src/ec/g2.ts (1a697/a9dea), off-circuit comment added to G2Line.fromPoints in src/lines/index.ts (a9dea), no changes to toBigInt usage in witness blocks or off-circuit code (1a697), changelog updated

Response

Every use of `Field.toBigInt()` in proof-conversion is idiomatic and correctly constrained. Inside `Provable.witness` callbacks the extracted value is constrained back in-circuit via `assertEquals`, `assertMul`, etc. All remaining call sites are off-circuit (deepClone, witness trackers, proof parsing). This follows the standard witness pattern documented by o1Labs.

As confirmed by o1Labs, `Field.toBigInt()` is fine to use in witness blocks and off-circuit code for production systems.

The "debug-only" warning in the `Field.toBigInt()` docstring is heavy-handed. o1js itself enforces the safety boundary at compile time, calling `Field.toBigInt()` on a variable inside a ZkProgram method throws:

```
Error: x.toBigInt() was called on a variable field element `x` in provable code.
This is not supported, because variables represent an abstract computation,
which only carries actual values during proving, but not during compiling.
```

It is not possible to misuse `Field.toBigInt()` inside provable code; the library prevents it. We also note that `ForeignField.toBigInt()` does not carry any such warning, its docstring is simply "Convert this field element to a bigint" - which further suggests the warning is an inconsistency in the o1js API documentation rather than a genuine safety boundary.

We accept the recommendation to remove dead code and add comments where appropriate.

Commit - Fix applied

- `src/ec/g2.ts`: removed dead code `G2Affine.add` method (1a697/a9dea).

Finding 3.17 [a9dea]

- **Report title:** Undocumented value-dependence for circuit-generating functions
- **Severity:** Informational · **Likelihood:** N/A · **Impact:** Informational
- **Target:** `proof-conversion`
- **Reviewed by Zellic:** No

Issue

o1js circuits are traced once at compile time, so a JavaScript `if` does not produce an in-circuit branch. Several helpers take circuit-typed parameters but choose between code paths with a JS `if`, which their signatures do not make clear. No current impact, as no caller expects in-circuit branching.

Fix branch and commits

proof-conversion · branch `proof-conversion/FIX/audit-1a697-a9dea` · commits list

- C1+C2 `f01d3c9a` · 2026-07-02 · **cited in report**

> FIX: audit 1a697 and a9dea, dead code G2Affine.add removed from src/ec/g2.ts (1a697/a9dea), off-circuit comment added to G2Line.fromPoints in src/lines/index.ts (a9dea), no changes to toBigInt usage in witness blocks or off-circuit code (1a697), changelog updated

Response

We accept the recommendation to remove dead code and add comments where appropriate.

Commit - Fix applied

- `src/ec/g2.ts`: removed dead code `G2Affine.add` method (1a697/a9dea).
- `src/lines/index.ts`: added comment to `G2Line.fromPoints` documenting that the `eq.toBigInt()` branch is resolved at JS level and is off-circuit only (a9dea).

Finding 3.18 [adcd3]

- **Report title:** Groth16 proof parser does not validate that piN keys are sequential
- **Severity:** Informational · **Likelihood:** N/A · **Impact:** Informational
- **Target:** `proof-conversion::groth::proof`
- **Reviewed by Zellic:** No

Issue

The Groth16 proof parser counts how many of `pi1`..`pi6` are present but does not check they are contiguous. All known producers emit sequential keys, so this is about clearer failures for malformed or hand-made inputs.

Fix branch and commits

proof-conversion · branch `proof-conversion/FIX/audit-adcd3` · commits list

- C1 `93015b57` · 2026-06-02

> CHORE: audit adcd3 commit 1, regression tests exposing missing piN contiguity, icN contiguity, and exact-shape validation in crypto guards, proof spec 19 fail 5 pass, vk spec 10 fail 14 pass, changelog updated.

- C2 `4209fb0b` · 2026-06-02 · **cited in report**

> FIX: audit adcd3 commit 2, assertExactStructure and piN/icN contiguity checks added to proof and vk parsers, crypto guards patched to reject extra keys, optional field support added to assertExactStructure, all regression tests pass (48 pass 0 fail), changelog updated.

Response

The finding is acknowledged. In the course of addressing it, scope was expanded to cover `GrothVk.parse` (`src/groth/vk.ts`) which has the same class of issue for `icN` keys - the VK parser collects whatever `ic0` to `ic6` keys are present without checking contiguity, and a FIXME comment in the code (`// FIXME CHECKME what if we have skipped some??`) confirms this was already known. Both parsers will be fixed together: `assertExactStructure` added for schema validation and an explicit contiguity check for sequential key ordering.

In the course of writing tests, three further bugs were identified in the validation layer:

- `isAffinePoint2d` (`src/api/validation/guards/crypto.ts`): used `'x' in obj && 'y' in obj` without checking key count, so `{ x: '1', y: '2', extra: 'bad' }` passed validation.
- `isComplexAffinePoint2d` (`src/api/validation/guards/crypto.ts`): same issue - checked for presence of the four expected keys but not exclusivity.
- `isField12` (`src/api/validation/guards/crypto.ts`): same issue - checked all 12 keys were present but did not reject objects with additional keys.

An additional limitation was identified in the validation layer:

- `assertExactStructure` (`src/api/validation/validation.ts`): treated all schema keys as required, so optional fields (wrapped with `isOptionalField`) could not be absent from the validated object - this was required to support optional `piN` and `icN` fields in the proof and VK schemas.

All four will be fixed in commit 2 alongside the parser fixes.

Commit 1 - Regression tests exposing missing contiguity and exact-shape validation

- **Regression tests** (`src/groth/adcd3_regression.spec.ts`): 24 tests covering `detectInputCountFromProof` - valid contiguous sequences (0 through 6 inputs), non-contiguous sequences (gaps at various positions), schema violations (missing/wrong-type fields), and unknown fields (pi0, pi7, extra keys on top-level and nested objects including negA, C, B). On unpatched code 19 fail, 5 pass.
- **Regression tests** (`src/groth/adcd3_vk_regression.spec.ts`): 24 tests covering `GrothVk.parse` - valid contiguous ic sequences (ic0 through ic3), non-contiguous sequences, missing required fields, wrong

field types, and unknown fields (top-level and nested: ic0, delta, gamma, alpha_beta, w27). On unpatched code 10 fail, 14 pass.

Run: `npm run test:jest -- src/groth/adcd3_regression.spec.ts`

Run: `npm run test:jest -- src/groth/adcd3_vk_regression.spec.ts`

Results:

- Proof regression tests: 19 fail, 5 pass. Contiguity, schema, optional-field, and exact-shape checks all absent.
- VK regression tests: 10 fail, 14 pass. Contiguity, optional-field, and exact-shape checks absent; basic missing-field checks already present in current code.

Commit 2 - Fix applied

- **`src/groth/proof.ts`**: `detectInputCountFromProof` updated with `assertExactStructure` schema validation and explicit `piN` contiguity check. `isO1jsProof` schema defined using `isAffinePoint2d`, `isComplexAffinePoint2d`, and `isOptionalString`.
- **`src/groth/vk.ts`**: `GrothVk.parse` updated with `assertExactStructure` schema validation and explicit `icN` contiguity check. `isGrothVk` schema defined using `isAffinePoint2d`, `isComplexAffinePoint2d`, `isField12`, and `isOptionalAffinePoint2d`. FIXME comment removed.
- **`src/api/validation/validation.ts`**: `assertExactStructure` patched to treat missing keys as valid when the schema validator accepts `undefined`, enabling optional field support.
- **`src/api/validation/guards/crypto.ts`**: `isAffinePoint2d` and `isComplexAffinePoint2d` updated to check exact key count in addition to key presence. `isField12` updated to check `Object.keys(obj).length === 12`. `isOptionalAffinePoint2d` added.
- **`src/api/validation/guards/strings.ts`**: New file. `isOptionalString` defined here with docstring.
- **`src/api/validation/guards/index.ts`**: `strings.ts` added to exports.

Results:

- Proof regression tests: 24 pass, 0 fail.
- VK regression tests: 24 pass, 0 fail.
- Existing validation tests (`src/api/validation/guards/test.spec.ts`): 14 pass, 0 fail.

Finding 3.19 [af97e]

- **Report title:** Usage of deprecated Gadgets.SHA256 API
- **Severity:** Informational · **Likelihood:** N/A · **Impact:** Informational
- **Target:** `proof-conversion::plonk::fiat-shamir::Sp1PlonkFiatShamir, proof-conversion::plonk::piop::hash_fr`
- **Reviewed by Zellic:** No

Issue

The PLONK Fiat-Shamir code uses `Gadgets.SHA256`, deprecated since o1js v2.3.0 in favour of `Gadgets.SHA2`. It may stop being maintained or be removed.

Fix branch and commits

proof-conversion · branch `proof-conversion/FIX/audit-af97e` · commits list

- C1+C2 `b2406ef6` · 2026-07-01 · **cited in report**

> FIX: audit af97e, deprecated Gadgets.SHA256 and Hash.SHA2_256 replaced with Gadgets.SHA2 across all call sites (fiat-shamir, hash_fr, batcher, sha_hash, parse_pi), unused Hash imports removed, changelog updated

Response

We agree that the deprecated method should be replaced. We had planned to remove all `Gadgets.SHA256` references before the audit began, and recorded the deprecation under "Outstanding" in the changelog in February 2026. The removal was not straightforward: the code uses the primitive's internal functions (`initialState`, `createMessageSchedule`, `compression`) to transfer SHA-256 state between zkp7 and zkp8, and an earlier attempt to swap to `Gadgets.SHA2` produced unexpected differences in the output, so we deferred the migration.

In January 2026, o1Labs recommended `Hash.SHA2_256` as the replacement. However, it exposes only `.hash(data)`, with no way to supply a mid-stream state, run one block at a time or obtain the initial state. And although `Hash.SHA2_256` is not marked as deprecated, its docstring indicates that it is an alias for `Gadgets.SHA256.hash`, which is deprecated, so it would not have resolved the deprecation. In February 2026, o1Labs confirmed that they do not plan any further deprecations, including for `Gadgets.SHA2`, which made it a suitable replacement.

Commit - Fix applied

- **`src/plonk/fiat-shamir/index.ts`**: `Gadgets.SHA256.initialState`, `Gadgets.SHA256.createMessageSchedule`, `Gadgets.SHA256.compression` replaced with `Gadgets.SHA2.initialState<UInt32>(256)`, `Gadgets.SHA2.messageSchedule(256, ...)`, `Gadgets.SHA2.compression(256, ...)`. `Hash.SHA2_256.hash` calls replaced with `Gadgets.SHA2.hash(256, ...)` as `Hash.SHA2_256` aliases the deprecated `Gadgets.SHA256.hash` in its implementation. Unused `Hash` import removed.
- **`src/plonk/piop/hash_fr.ts`**: `Hash.SHA2_256.hash` calls replaced with `Gadgets.SHA2.hash(256, ...)` for the same reason. Unused `Hash` import removed.
- **`src/plonk/parse_pi.ts`**: deprecated comment and commented-out `Gadgets.SHA256.hash` call removed.
- **`src/blobstream/batcher.ts`**: `Gadgets.SHA256.initialState`, `Gadgets.SHA256.createMessageSchedule`, `Gadgets.SHA256.compression` replaced with `Gadgets.SHA2` equivalents.
- **`src/sha/sha_hash.ts`**: `Gadgets.SHA256.hash`, `Gadgets.SHA256.initialState`, `Gadgets.SHA256.createMessageSchedule`, `Gadgets.SHA256.compression` replaced with `Gadgets.SHA2` equivalents.

Finding 3.20 [d7ffa]

- **Report title:** Usage of @internal o1js APIs
- **Severity:** Informational · **Likelihood:** N/A · **Impact:** Informational
- **Target:** `proof-conversion::towers::assertMul`
- **Reviewed by Zellic:** No — resolved upstream (see Findings closed with rationale)

Issue

`assertMul` and `UnreducedSum` rely on `Gadgets.ForeignField.assertMul` and `Gadgets.ForeignField.Sum`, which o1js tagged `@internal`. A silent change upstream could alter the constraint system without a test failing. o1Labs has since removed the `@internal` tags, and the APIs are public from o1js 3.0.0. See Findings closed with rationale.

Fix branch and commits

No fix branch or fix commit exists. o1Labs resolved the finding upstream in o1js 3.0.0.

Response

We acknowledge the finding. In our next release of proof-conversion, we will upgrade directly to o1js 3.1.0, the latest release, which includes this change. See Findings closed with rationale.

Finding 3.21 [b8891]

- **Report title:** Constructor AffineCache assigns yp_prime twice
- **Severity:** Informational · **Likelihood:** N/A · **Impact:** Informational
- **Target:** `proof-conversion::lines::AffineCache`
- **Reviewed by Zellic:** No

Issue

The `AffineCache` constructor assigns `yp_prime` twice; the first (an inversion plus canonical check) is immediately overwritten. Harmless apart from wasted constraints.

Fix branch and commits

proof-conversion · branch `proof-conversion/FIX/audit-b8891` · commits list

- C1+C2 `8b6b0d40` · 2026-06-23 · **cited in report**

> FIX: audit b8891, redundant first assignment to yp_prime removed from AffineCache constructor (src/lines/precompute.ts), changelog updated

Response

We acknowledge the finding. The first assignment on line 15 computes `p.y.inv().assertCanonical()` and is immediately overwritten by the `Provable.witness` path on lines 16-18. The witnessed inverse is then properly constrained by the `mul(p.y).assertEquals(1n)` check on line 19. The first assignment is dead code that produces unnecessary `FpC.inv()` + `assertCanonical()` gates whose output is discarded.

AffineCache is constructed at 30 call sites across Groth16 (zkp0-zkp6, accumulate_lines: 3 per site for negA, C, PI) and PLONK (zkp13-zkp16, accumulate_lines: 2 per site for A, negB), so the wasted gates are multiplied across the full recursion pipeline.

Commit - Fix applied

- `src/lines/precompute.ts`: removed redundant first assignment `this.yp_prime = p.y.inv().assertCanonical()` on line 15.

Finding 3.22 [e68c5]

- **Report title:** Function G2Line.evaluate_g1 is dead code and does not implement the correct evaluation of the line function on G1
- **Severity:** Informational · **Likelihood:** N/A · **Impact:** Informational
- **Target:** `proof-conversion::lines`
- **Reviewed by Zellic:** No

Issue

`G2Line.evaluate_g1` is unused and does not correctly evaluate the line function on G1 (the correct version is `psi`). Risk is someone using it by mistake later.

Fix branch and commits

proof-conversion · branch `proof-conversion/FIX/audit-e68c5` · commits list

- C1+C2 `47fcd6d4` · 2026-06-23 · **cited in report**

> FIX: audit e68c5, dead code evaluate_g1 method removed from G2Line (src/lines/index.ts), unused G1Affine import removed, changelog updated

Response

We acknowledge the finding. `evaluate_g1` has zero call sites in the codebase. The method naively substitutes G1 coordinates into the G2 line equation without applying the twist isomorphism, which `psi` handles correctly via `AffineCache`'s `xp_prime` and `yp_prime`. Leaving it in place risks incorrect use in future edits.

Commit - Fix applied

- `src/lines/index.ts`: removed `evaluate_g1` method from `G2Line` class.

Finding 3.23 [e4e27]

- **Report title:** Function prepare_consensus_mpt_proof_inputs reconstructs ExecutionHttpProxy from env on every window
- **Severity:** Informational · **Likelihood:** N/A · **Impact:** Informational
- **Target:** `bridge-head::nori::rpcs::consensus`
- **Reviewed by Zellic:** No

Issue

`prepare_consensus_mpt_proof_inputs` builds a fresh `ExecutionHttpProxy` from environment variables every proving window. Performance only.

Fix branch and commits

nori-bridge-head · branch `nori-bridge-head/FIX/audit-e4e27` · commits list

- C1+C2 `784e70bd` · 2026-06-15 · **cited in report**

> FIX: Audit e4e27, ExecutionHttpProxy is now a field on ConsensusHttpProxy constructed once in from_env() instead of being reconstructed via try_from_env() on every proving window, ConsensusHttpProxy itself is now constructed once at startup in api.rs and passed into start_validated_consensus_finality_change_detector and the validation actor instead of being reconstructed per job in the loop, no from_env calls remain in any loop or per-window path, changelog documents the verbatim finding, response and commit details.

Response

We agree with the finding. The FIXME comment on the line above the call site acknowledged this was untidy. The `from_env()` call is cheap (env var reads, URL parsing, HTTP provider construction, no network calls) and the cost per window is negligible, but reconstructing identical config on every invocation is unnecessary.

The suggested approach of owning `ExecutionHttpProxy` as a field on `ConsensusHttpProxy` was adopted. `ConsensusHttpProxy::from_env()` now also constructs the `ExecutionHttpProxy` and stores it as a field, so `prepare_consensus_mpt_proof_inputs` uses `self.execution_proxy` instead of calling `try_from_env()`.

While auditing all `from_env` call sites, the same pattern was found in `validate_and_prepare_proof_inputs_actor` (`finality_change_detector.rs`), where `ConsensusHttpProxy::try_from_env()` was called inside the job loop on every proving window. This was hoisted above the loop. The `ConsensusHttpProxy` is now constructed once in `api.rs` at startup alongside the `ProverConfig`, passed into `start_validated_consensus_finality_change_detector`, which passes it into the validation actor. No `from_env` calls remain in any loop or per-window path.

Commit

- **`ConsensusHttpProxy`** (`nori/src/rpcs/consensus/mod.rs`): added `execution_proxy: ExecutionHttpProxy<S>` field to the struct, constructed in `from_env()`. `prepare_consensus_mpt_proof_inputs` now uses `self.execution_proxy` instead of `ExecutionHttpProxy::try_from_env()`, removing the FIXME.
- **`validate_and_prepare_proof_inputs_actor`** (`nori/src/bridge_head/finality_change_detector.rs`): changed signature to accept a `ConsensusHttpProxy` parameter instead of constructing one internally. Removed per-job `try_from_env()` calls from both the dual-window and solo-window branches.
- **`start_validated_consensus_finality_change_detector`** (`nori/src/bridge_head/finality_change_detector.rs`): changed signature to accept a `ConsensusHttpProxy` parameter. Uses it for the initial `get_latest_finality_slot()` call and passes it into the validation actor. Removed `MainnetConsensusSpec` and `HttpRpc` imports that are no longer needed.
- **`BridgeHead::run`** (`nori/src/bridge_head/api.rs`): constructs `ConsensusHttpProxy` once at startup alongside `ProverConfig` and passes it into the finality change detector.